stv_v2

Generated by Doxygen 1.9.1

1 STV2 - StraTegic Verifier 2 1
1.1 Usage . . . 1
1.2 Tests . . . 1
1.3 Performance estimation . . . 2
1.4 Specification . . . 2
1.5 Examples and templates . . . 3
1.6 Misc . . . 3

2 Hierarchical Index 5
2.1 Class Hierarchy . . . 5

3 Class Index 7
3.1 Class List . . . 7

4 File Index 11
4.1 File List . . . 11

5 Class Documentation 15
5.1 Action Struct Reference . . . 15
5.1.1 Detailed Description . . . 15
5.2 ActionTemplate Class Reference . . . 15
5.2.1 Detailed Description . . . 16
5.2.2 Constructor & Destructor Documentation . . . 16
5.2.2.1 ActionTemplate() . . . 16
5.3 Agent Class Reference . . . 16
5.3.1 Detailed Description . . . 17
5.3.2 Constructor & Destructor Documentation . . . 17
5.3.2.1 Agent() . . . 17
5.3.3 Member Function Documentation . . . 17
5.3.3.1 includesState() . . . 17
5.4 AgentTemplate Class Reference . . . 18
5.4.1 Detailed Description . . . 18
5.4.2 Member Function Documentation . . . 19
5.4.2.1 addInitial() . . . 19
5.4.2.2 addLocal() . . . 19
5.4.2.3 addPersistent() . . . 20
5.4.2.4 addTransition() . . . 20
5.4.2.5 generateAgent() . . . 21
5.4.2.6 setIdent() . . . 21
5.4.2.7 setInitState() . . . 22
5.5 Assignment Class Reference . . . 23
5.5.1 Detailed Description . . . 23
5.5.2 Constructor & Destructor Documentation . . . 23
5.5.2.1 Assignment() . . . 23

5.5.3 Member Function Documentation . . . 24
5.5.3.1 assign() . . . 24
5.6 Cfg Struct Reference . . . 24
5.7 Condition Struct Reference . . . 25
5.7.1 Detailed Description . . . 25
5.8 DecisionEntry Struct Reference . . . 25
5.9 DotGraph Class Reference . . . 26
5.9.1 Member Function Documentation . . . 27
5.9.1.1 addEdge() . . . 27
5.9.1.2 addNode() . . . 27
5.9.1.3 saveToFile() . . . 27
5.9.2 Member Data Documentation . . . 28
5.9.2.1 styleString . . . 28
5.10 EpistemicClass Struct Reference . . . 28
5.10.1 Detailed Description . . . 28
5.11 ExprAdd Class Reference . . . 29
5.11.1 Detailed Description . . . 29
5.11.2 Constructor & Destructor Documentation . . . 29
5.11.2.1 ExprAdd() . . . 29
5.11.3 Member Function Documentation . . . 29
5.11.3.1 eval() . . . 29
5.12 ExprAnd Class Reference . . . 31
5.12.1 Detailed Description . . . 31
5.12.2 Constructor & Destructor Documentation . . . 31
5.12.2.1 ExprAnd() . . . 31
5.12.3 Member Function Documentation . . . 32
5.12.3.1 eval() . . . 32
5.13 ExprConst Class Reference . . . 32
5.13.1 Detailed Description . . . 33
5.13.2 Constructor & Destructor Documentation . . . 33
5.13.2.1 ExprConst() . . . 33
5.13.3 Member Function Documentation . . . 33
5.13.3.1 eval() . . . 33
5.14 ExprDiv Class Reference . . . 34
5.14.1 Detailed Description . . . 34
5.14.2 Constructor & Destructor Documentation . . . 34
5.14.2.1 ExprDiv() . . . 34
5.14.3 Member Function Documentation . . . 34
5.14.3.1 eval() . . . 34
5.15 ExprEq Class Reference . . . 36
5.15.1 Detailed Description . . . 36
5.15.2 Constructor & Destructor Documentation . . . 36

5.15.2.1 ExprEq() . . . 36
5.15.3 Member Function Documentation . . . 37
5.15.3.1 eval() . . . 37
5.16 ExprGe Class Reference . . . 37
5.16.1 Detailed Description . . . 38
5.16.2 Constructor & Destructor Documentation . . . 38
5.16.2.1 ExprGe() . . . 38
5.16.3 Member Function Documentation . . . 38
5.16.3.1 eval() . . . 38
5.17 ExprGt Class Reference . . . 39
5.17.1 Detailed Description . . . 39
5.17.2 Constructor & Destructor Documentation . . . 39
5.17.2.1 ExprGt() . . . 39
5.17.3 Member Function Documentation . . . 39
5.17.3.1 eval() . . . 39
5.18 ExprHart Class Reference . . . 41
5.18.1 Detailed Description . . . 41
5.18.2 Constructor & Destructor Documentation . . . 41
5.18.2.1 ExprHart() . . . 41
5.18.3 Member Function Documentation . . . 42
5.18.3.1 eval() . . . 42
5.19 ExprIdent Class Reference . . . 42
5.19.1 Detailed Description . . . 43
5.19.2 Constructor & Destructor Documentation . . . 43
5.19.2.1 ExprIdent() . . . 43
5.19.3 Member Function Documentation . . . 43
5.19.3.1 eval() [1/2] . . . 43
5.19.3.2 eval() [2/2] . . . 43
5.20 ExprKnow Class Reference . . . 45
5.20.1 Detailed Description . . . 45
5.20.2 Constructor & Destructor Documentation . . . 45
5.20.2.1 ExprKnow() . . . 45
5.20.3 Member Function Documentation . . . 46
5.20.3.1 eval() . . . 46
5.21 ExprLe Class Reference . . . 46
5.21.1 Detailed Description . . . 47
5.21.2 Constructor & Destructor Documentation . . . 47
5.21.2.1 ExprLe() . . . 47
5.21.3 Member Function Documentation . . . 47
5.21.3.1 eval() . . . 47
5.22 ExprLt Class Reference . . . 48
5.22.1 Detailed Description . . . 48

5.22.2 Constructor & Destructor Documentation . . . 48
5.22.2.1 ExprLt() . . . 48
5.22.3 Member Function Documentation . . . 48
5.22.3.1 eval() . . . 48
5.23 ExprMul Class Reference . . . 50
5.23.1 Detailed Description . . . 50
5.23.2 Constructor & Destructor Documentation . . . 50
5.23.2.1 ExprMul() . . . 50
5.23.3 Member Function Documentation . . . 51
5.23.3.1 eval() . . . 51
5.24 ExprNe Class Reference . . . 51
5.24.1 Detailed Description . . . 52
5.24.2 Constructor & Destructor Documentation . . . 52
5.24.2.1 ExprNe() . . . 52
5.24.3 Member Function Documentation . . . 52
5.24.3.1 eval() . . . 52
5.25 ExprNode Class Reference . . . 53
5.25.1 Detailed Description . . . 53
5.25.2 Member Function Documentation . . . 53
5.25.2.1 eval() . . . 53
5.26 ExprNot Class Reference . . . 54
5.26.1 Detailed Description . . . 54
5.26.2 Constructor & Destructor Documentation . . . 54
5.26.2.1 ExprNot() . . . 54
5.26.3 Member Function Documentation . . . 54
5.26.3.1 eval() . . . 54
5.27 ExprOr Class Reference . . . 55
5.27.1 Detailed Description . . . 55
5.27.2 Constructor & Destructor Documentation . . . 55
5.27.2.1 ExprOr() . . . 55
5.27.3 Member Function Documentation . . . 56
5.27.3.1 eval() . . . 56
5.28 ExprRem Class Reference . . . 56
5.28.1 Detailed Description . . . 57
5.28.2 Constructor & Destructor Documentation . . . 57
5.28.2.1 ExprRem() . . . 57
5.28.3 Member Function Documentation . . . 57
5.28.3.1 eval() . . . 57
5.29 ExprSub Class Reference . . . 58
5.29.1 Detailed Description . . . 58
5.29.2 Constructor & Destructor Documentation . . . 58
5.29.2.1 ExprSub() . . . 58

5.29.3 Member Function Documentation . . . 58
5.29.3.1 eval() . . . 58
5.30 Formula Struct Reference . . . 60
5.30.1 Detailed Description . . . 60
5.31 FormulaTemplate Struct Reference . . . 60
5.31.1 Detailed Description . . . 61
5.32 GlobalModel Struct Reference . . . 61
5.32.1 Detailed Description . . . 61
5.33 GlobalModelGenerator Class Reference . . . 62
5.33.1 Detailed Description . . . 64
5.33.2 Member Function Documentation . . . 64
5.33.2.1 computeEpistemicClassHash() . . . 64
5.33.2.2 computeGlobalStateHash() . . . 64
5.33.2.3 createIterativeStrategy() . . . 64
5.33.2.4 expandAndReduceAllStates() . . . 65
5.33.2.5 expandState() . . . 65
5.33.2.6 expandStateAndReturn() . . . 65
5.33.2.7 findGlobalStateInEpistemicClass() . . . 66
5.33.2.8 findOrCreateEpistemicClass() . . . 66
5.33.2.9 findOrCreateEpistemicClassForKnowledge() . . . 66
5.33.2.10 generateGlobalTransitions() . . . 67
5.33.2.11 generateInitState() . . . 67
5.33.2.12 generateStateFromLocalStates() . . . 67
5.33.2.13 getActionNameFromStateInStrategy() . . . 68
5.33.2.14 getAgentInstanceByName() . . . 68
5.33.2.15 getCoalitionIdentifier() . . . 69
5.33.2.16 getCurrentGlobalModel() . . . 69
5.33.2.17 getFormula() . . . 69
5.33.2.18 getFormulaCorectness() . . . 69
5.33.2.19 getFormulaSize() . . . 70
5.33.2.20 initModel() . . . 70
5.33.2.21 initStrategy() . . . 70
5.33.2.22 nextIterativeStrategy() . . . 70
5.34 GlobalState Struct Reference . . . 71
5.34.1 Detailed Description . . . 72
5.34.2 Member Function Documentation . . . 72
5.34.2.1 getGlobalStateEnvironment() . . . 72
5.34.2.2 toString() . . . 72
5.35 GlobalModelGenerator::GlobalStateTupleHash Struct Reference . . . 72
5.36 GlobalTransition Struct Reference . . . 73
5.36.1 Detailed Description . . . 73
5.36.2 Member Function Documentation . . . 73

5.36.2.1 joinLocalTransitionNames() . . . 73
5.37 HistoryDbg Class Reference . . . 74
5.37.1 Detailed Description . . . 74
5.37.2 Member Function Documentation . . . 74
5.37.2.1 addEntry() . . . 74
5.37.2.2 cloneEntry() . . . 75
5.37.2.3 markEntry() . . . 75
5.37.2.4 print() . . . 75
5.38 HistoryEntry Struct Reference . . . 76
5.38.1 Detailed Description . . . 76
5.38.2 Member Function Documentation . . . 76
5.38.2.1 toString() . . . 77
5.39 IterativeVerif Class Reference . . . 77
5.40 LocalModels Struct Reference . . . 77
5.40.1 Detailed Description . . . 78
5.41 LocalState Class Reference . . . 78
5.41.1 Detailed Description . . . 78
5.41.2 Member Function Documentation . . . 78
5.41.2.1 compare() . . . 78
5.41.2.2 toString() . . . 79
5.42 LocalStateTemplate Class Reference . . . 79
5.42.1 Detailed Description . . . 80
5.43 LocalTransition Struct Reference . . . 80
5.43.1 Detailed Description . . . 80
5.44 ModelParser Class Reference . . . 81
5.44.1 Detailed Description . . . 81
5.44.2 Member Function Documentation . . . 81
5.44.2.1 parse() . . . 81
5.44.2.2 parseAndOverwriteFormula() . . . 81
5.45 ProbabilityEntry Class Reference . . . 82
5.45.1 Detailed Description . . . 82
5.46 ProbabilityStrategyDecisions Class Reference . . . 83
5.46.1 Detailed Description . . . 83
5.47 ProbabilityTrueFalse Struct Reference . . . 83
5.47.1 Detailed Description . . . 83
5.48 ProbAdd Class Reference . . . 83
5.48.1 Detailed Description . . . 84
5.48.2 Constructor & Destructor Documentation . . . 84
5.48.2.1 ProbAdd() . . . 84
5.48.3 Member Function Documentation . . . 84
5.48.3.1 eval() . . . 84
5.49 ProbConst Class Reference . . . 85

5.49.1 Detailed Description . . . 85
5.49.2 Constructor & Destructor Documentation . . . 85
5.49.2.1 ProbConst() . . . 85
5.49.3 Member Function Documentation . . . 86
5.49.3.1 eval() . . . 86
5.50 ProbDiv Class Reference . . . 86
5.50.1 Detailed Description . . . 86
5.50.2 Constructor & Destructor Documentation . . . 87
5.50.2.1 ProbDiv() . . . 87
5.50.3 Member Function Documentation . . . 87
5.50.3.1 eval() . . . 87
5.51 ProbMul Class Reference . . . 87
5.51.1 Detailed Description . . . 88
5.51.2 Constructor & Destructor Documentation . . . 88
5.51.2.1 ProbMul() . . . 88
5.51.3 Member Function Documentation . . . 88
5.51.3.1 eval() . . . 88
5.52 ProbNode Class Reference . . . 89
5.52.1 Detailed Description . . . 89
5.52.2 Member Function Documentation . . . 89
5.52.2.1 eval() . . . 89
5.53 ProbSub Class Reference . . . 90
5.53.1 Detailed Description . . . 90
5.53.2 Constructor & Destructor Documentation . . . 90
5.53.2.1 ProbSub() . . . 90
5.53.3 Member Function Documentation . . . 90
5.53.3.1 eval() . . . 90
5.54 Result Struct Reference . . . 91
5.55 StateVerificationInfo Struct Reference . . . 91
5.55.1 Detailed Description . . . 92
5.56 StrategyBitsComparator Struct Reference . . . 92
5.56.1 Detailed Description . . . 92
5.57 StrategyCollection Class Reference . . . 92
5.57.1 Detailed Description . . . 92
5.58 StrategyCollectionTemplate Class Reference . . . 93
5.58.1 Member Function Documentation . . . 93
5.58.1.1 addAction() . . . 93
5.59 StrategyEntry Struct Reference . . . 93
5.60 StrategyParser Class Reference . . . 94
5.60.1 Detailed Description . . . 94
5.60.2 Member Function Documentation . . . 94
5.60.2.1 parse() . . . 94

5.61 TestVerif Class Reference . . . 95
5.62 TransitionTemplate Class Reference . . . 95
5.62.1 Detailed Description . . . 96
5.62.2 Constructor & Destructor Documentation . . . 96
5.62.2.1 TransitionTemplate() . . . 96
5.63 Var Struct Reference . . . 96
5.63.1 Detailed Description . . . 97
5.64 Verification Class Reference . . . 97
5.64.1 Detailed Description . . . 99
5.64.2 Constructor & Destructor Documentation . . . 99
5.64.2.1 Verification() . . . 99
5.64.3 Member Function Documentation . . . 100
5.64.3.1 addHistoryContext() . . . 100
5.64.3.2 addHistoryDecision() . . . 100
5.64.3.3 addHistoryMarkDecisionAsInvalid() . . . 101
5.64.3.4 addHistoryStateStatus() . . . 101
5.64.3.5 addHistoryUncontrolledDecision() . . . 101
5.64.3.6 areGlobalStatesInTheSameEpistemicClass() . . . 102
5.64.3.7 checkUncontrolledSet() . . . 102
5.64.3.8 equivalentGlobalTransitions() . . . 102
5.64.3.9 getEpistemicClassForGlobalState() . . . 103
5.64.3.10 getNaturalStrategy() . . . 103
5.64.3.11 getReducedStrategy() . . . 103
5.64.3.12 getStrategyComplexity() . . . 104
5.64.3.13 globalStateToValueBits() . . . 104
5.64.3.14 increaseProbability() . . . 104
5.64.3.15 isAgentInCoalition() . . . 104
5.64.3.16 isGlobalTransitionControlledByCoalition() . . . 105
5.64.3.17 lowerProbability() . . . 105
5.64.3.18 newHistoryMarkDecisionAsInvalid() . . . 106
5.64.3.19 printCurrentHistory() . . . 106
5.64.3.20 reduceStrategy() . . . 106
5.64.3.21 restoreHistory() . . . 107
5.64.3.22 revertLastDecision() . . . 107
5.64.3.23 undoHistoryUntil() . . . 108
5.64.3.24 undoLastHistoryEntry() . . . 108
5.64.3.25 verify() . . . 108
5.64.3.26 verifyGlobalState() . . . 108
5.64.3.27 verifyLocalStates() . . . 109
5.64.3.28 verifyStrategy() . . . 109
5.64.3.29 verifyTransitionSets() . . . 109
5.65 VerificationIterative Class Reference . . . 110

5.65.1 Detailed Description . . . 112
5.65.2 Constructor & Destructor Documentation . . . 112
5.65.2.1 VerificationIterative() . . . 112
5.65.3 Member Function Documentation . . . 112
5.65.3.1 addHistoryAnswerProbability() . . . 112
5.65.3.2 addHistoryContext() . . . 113
5.65.3.3 addHistoryDecision() . . . 113
5.65.3.4 addHistoryMarkDecisionAsInvalid() . . . 114
5.65.3.5 addHistoryStateStatus() . . . 114
5.65.3.6 areGlobalStatesInTheSameEpistemicClass() . . . 114
5.65.3.7 checkIfProbabilistic() . . . 115
5.65.3.8 dissolveLoopedProbabilityTransition() . . . 115
5.65.3.9 equivalentGlobalTransitions() . . . 115
5.65.3.10 findLoopedProbabilityTransitions() . . . 116
5.65.3.11 getEpistemicClassForGlobalState() . . . 116
5.65.3.12 getNaturalStrategy() . . . 116
5.65.3.13 getReducedStrategy() . . . 117
5.65.3.14 getStrategyComplexity() . . . 117
5.65.3.15 globalStateToValueBits() . . . 117
5.65.3.16 isAgentInCoalition() . . . 118
5.65.3.17 isGlobalTransitionControlledByCoalition() . . . 118
5.65.3.18 reduceStrategy() . . . 118
5.65.3.19 revertToLastDecision() . . . 119
5.65.3.20 testForAndFixBadAgents() . . . 119
5.65.3.21 verify() . . . 119
5.65.3.22 verifyLocalStates() . . . 119
5.66 yy_buffer_state Struct Reference . . . 120
5.66.1 Member Data Documentation . . . 120
5.66.1.1 yy_bs_column . . . 120
5.66.1.2 yy_bs_lineno . . . 120
5.67 yy_trans_info Struct Reference . . . 121
5.68 yyalloc Union Reference . . . 121
5.69 YYSTYPE Union Reference . . . 121

6 File Documentation **123**
6.1 Agent.cpp File Reference . . . 123
6.1.1 Detailed Description . . . 123
6.2 Agent.hpp File Reference . . . 123
6.2.1 Detailed Description . . . 123
6.3 Common.hpp File Reference . . . 124
6.3.1 Detailed Description . . . 124
6.4 DotGraph.cpp File Reference . . . 124

6.4.1 Detailed Description . . . 124
6.5 DotGraph.hpp File Reference . . . 124
6.5.1 Detailed Description . . . 125
6.5.2 Enumeration Type Documentation . . . 125
6.5.2.1 DotGraphBase . . . 125
6.6 EpistemicClass.hpp File Reference . . . 125
6.6.1 Detailed Description . . . 125
6.7 expressions.cc File Reference . . . 126
6.7.1 Detailed Description . . . 126
6.8 expressions.hpp File Reference . . . 126
6.8.1 Detailed Description . . . 127
6.9 GlobalModel.hpp File Reference . . . 127
6.9.1 Detailed Description . . . 128
6.10 GlobalModelGenerator.cpp File Reference . . . 128
6.10.1 Detailed Description . . . 128
6.11 GlobalModelGenerator.hpp File Reference . . . 128
6.11.1 Detailed Description . . . 129
6.12 GlobalState.cpp File Reference . . . 129
6.12.1 Detailed Description . . . 129
6.13 GlobalState.hpp File Reference . . . 129
6.13.1 Detailed Description . . . 129
6.14 GlobalTransition.cpp File Reference . . . 129
6.14.1 Detailed Description . . . 130
6.15 GlobalTransition.hpp File Reference . . . 130
6.15.1 Detailed Description . . . 130
6.16 LocalState.cpp File Reference . . . 130
6.16.1 Detailed Description . . . 130
6.17 LocalState.hpp File Reference . . . 130
6.17.1 Detailed Description . . . 131
6.18 LocalTransition.hpp File Reference . . . 131
6.18.1 Detailed Description . . . 131
6.19 ModelParser.cc File Reference . . . 131
6.19.1 Detailed Description . . . 132
6.20 nodes.cc File Reference . . . 132
6.20.1 Detailed Description . . . 132
6.21 nodes.hpp File Reference . . . 132
6.21.1 Detailed Description . . . 132
6.22 strategyNodes.hpp File Reference . . . 133
6.22.1 Detailed Description . . . 133
6.23 StrategyParser.cc File Reference . . . 133
6.23.1 Detailed Description . . . 133
6.24 StrategyParser.hpp File Reference . . . 134

6.24.1 Detailed Description . . . 134
6.25 Types.hpp File Reference . . . 134
6.25.1 Detailed Description . . . 135
6.25.2 Enumeration Type Documentation . . . 135
6.25.2.1 ProbabilitySign . . . 135
6.26 TypesDependency.hpp File Reference . . . 135
6.26.1 Detailed Description . . . 136
6.26.2 Enumeration Type Documentation . . . 136
6.26.2.1 HistoryEntryType . . . 136
6.26.2.2 StrategyEntryType . . . 137
6.27 Utils.cpp File Reference . . . 137
6.27.1 Detailed Description . . . 138
6.27.2 Function Documentation . . . 138
6.27.2.1 agentToString() . . . 138
6.27.2.2 envToString() . . . 138
6.27.2.3 getLocalStatesSCC() . . . 139
6.27.2.4 localModelsToString() . . . 139
6.27.2.5 outputGlobalModel() . . . 139
6.27.2.6 tarjanVisit() . . . 140
6.28 Utils.hpp File Reference . . . 140
6.28.1 Function Documentation . . . 141
6.28.1.1 agentToString() . . . 141
6.28.1.2 envToString() . . . 141
6.28.1.3 getLocalStatesSCC() . . . 142
6.28.1.4 localModelsToString() . . . 142
6.28.1.5 outputGlobalModel() . . . 142
6.29 Verification.cpp File Reference . . . 143
6.29.1 Detailed Description . . . 143
6.29.2 Function Documentation . . . 143
6.29.2.1 dbgHistEnt() . . . 143
6.29.2.2 dbgVerifStatus() . . . 144
6.29.2.3 verStatusToStr() . . . 144
6.30 Verification.hpp File Reference . . . 144
6.30.1 Enumeration Type Documentation . . . 145
6.30.1.1 TraversalMode . . . 145
6.30.2 Function Documentation . . . 145
6.30.2.1 verStatusToStr() . . . 146
6.31 VerificationIterative.cpp File Reference . . . 146
6.31.1 Detailed Description . . . 146
6.31.2 Function Documentation . . . 146
6.31.2.1 verifStatusToStr() . . . 146
6.32 VerificationIterative.hpp File Reference . . . 147

6.32.1 Enumeration Type Documentation 147
6.32.1.1 GraphTraversalMode 147
6.32.2 Function Documentation 148
6.32.2.1 verifStatusToStr() 148

Index **149**

Chapter 1

STV2 - StraTegic Verifier 2

1.1 Usage

To run:
```
cd build
make clean
make
./stv
```

or
```
cd build
./build-run
```

Configuration file:
```
build/config.txt
```

CLI configuration overwrite:

```
# Input model
./stv --file PATH_TO_MODEL
./stv -f PATH_TO_MODEL
# Mode
./stv -m 0  #
./stv -m 1  # generate GlobalModel
./stv -m 2  # run verification
./stv -m 3  # same as 1 && 2
# Flags
# --OUTPUT_GLOBAL_MODEL      stdout data on global model (after expandAllStates)
# --OUTPUT_LOCAL_MODELS      stdout data on local models ()
# --OUTPUT_DOT_FILES         generate .dot files for agent templates, local and global models
# --ADD_EPSILON_TRANSITIONS  generate global models with epsilon transitions
# --OVERWRITE_FORMULA        replace the formula from the model file with a different one
# --COUNTEREXAMPLE           output counterexample path if the formula verification returns an ERR
# --REDUCE                   reduce the amount of states and transitions using a DFS-POR algorithm and
       select the first correct transition
# --REDUCE_ALL               reduce the amount of states and transitions using a DFS-POR algorithm and
       select all avaliable transitions
# --FIXPOINT                 enables fixpoint approximation
# --NATURAL_STRATEGY         generate a natural strategy for the given model
```

1.2 Tests

To run tests:
```
cd build
make clean
make sample_test
./sample_test
```

or
```
cd build
./build-test
```

To run larger tests:
```
cd build
make clean
make sample_test
./sample_test
```

or
```
cd build
./build-big-test
```

You might need to run
```
ulimit -s unlimited
```

beforehand

1.3 Performance estimation

Ubuntu/WSL:

```
# Minimal
> /usr/bin/time -f "%M\t%e" ./stv
# %M - maximum resident set size in KB
# %e - elapsed real time (wall clock) in seconds
# More detailed
> /usr/bin/time -f "time result\ncmd:%C\nreal %es\nuser %Us \nsys  %Ss \nmemory:%MKB \ncpu %P" ./stv
# %C   command line and arguments
# %e   elapsed real time (wall clock) in seconds
# %U   user time in seconds
# %S   system (kernel) time in seconds
# %M   maximum resident set size in KB
# %P   percent of CPU this job got
# Full (verbose)
> /usr/bin/time -v ./stv
```

1.4 Specification

The specification language was inspired by ISPL (Interpreted Systems Programming Language) from MCMAS.
The detailed syntax for the input format can be derived from ∗./src/reader/{parser.y,scanner.l}∗, which intrinsically make up an EBNF grammar.
For the most parts, it is simple enough to get intuition just from looking at example's source code and the program's output.

IMPORTANTS NOTES:

1. the (local) action names must be unique;
2. the transition relation (from the global model) should be serial;

1.5 Examples and templates

In ∗./examples∗ and ∗./tests/examples∗ there are several ready-to-use MAS specification files together with a proposed property (captured by ATL formula) for verification.

Often, we would want to reason about different (data-)configurations of the same system.
Using the templates we can parameterize the system specification, such that we only need to describe its dynamic behaviour.
A template can be fed with a configuration data to generate a concrete instance of a system.
Moreover, their use is independent from the tool: one can choose any templating engine (of the myriads available) or even write a custom one from the scratch.

Here, we utilize the `EJS` templating engine.
It has a CLI support, which is comes in handy for the tests/benchmarks that involve systems in multiple configurations.

```
# EJS feeds the data (as a list of key:val pairs) to the template file to generate the output:
> npm exec -- ejs TEMPLATE_FILE.ejs -i "{PARAM1:VAL1,PARAM2:VAL2,...}" -o OUTPUT_FILE.txt
# Possible generation query for the "trains":
> npm exec -- ejs Trains.ejs -i '{"N_TRAINS":3,"WITH_FORMULA":1}' -o 3Trains1Controller.txt
# Possible generation query for the "simple voting":
> npm exec -- ejs Simple_voting.ejs -i '{"N_VOTERS":2,"N_CANDIDATES":1,"WITH_FORMULA":0}' -o
      2Voters1Coercer1Candidate.txt
```

1.6 Misc

With `OUTPUT_DOT_FILES` flag the program outputs ∗.dot∗ files for templates, local and global models where:

- nodes are labelled with its location name (comma-separated for the global state)
- shared transitions are denoted by blue colour

Use Graphviz `(link)` to view in other format (eps, pdf, jpeg, etc.):

```
# Analogously for other formats
dot -Tpng lts_of_AGENT.dot > lts_of_AGENT.png
```

For the smaller graphs use *dot2png.sh* script, which converts all ∗.dot∗ files from a current folder to ∗.png∗.
For bigger ones use `svg` format (may be viewed in Inkscape) and *dot2svg.sh*.

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Action . . . 15
ActionTemplate . . . 15
Agent . . . 16
AgentTemplate . . . 18
Assignment . . . 23
Cfg . . . 24
Condition . . . 25
DecisionEntry . . . 25
DotGraph . . . 26
EpistemicClass . . . 28
ExprNode . . . 53
- ExprAdd . . . 29
- ExprAnd . . . 31
- ExprConst . . . 32
- ExprDiv . . . 34
- ExprEq . . . 36
- ExprGe . . . 37
- ExprGt . . . 39
- ExprHart . . . 41
- ExprIdent . . . 42
- ExprKnow . . . 45
- ExprLe . . . 46
- ExprLt . . . 48
- ExprMul . . . 50
- ExprNe . . . 51
- ExprNot . . . 54
- ExprOr . . . 55
- ExprRem . . . 56
- ExprSub . . . 58

Formula . . . 60
FormulaTemplate . . . 60
GlobalModel . . . 61
GlobalModelGenerator . . . 62
GlobalState . . . 71
GlobalModelGenerator::GlobalStateTupleHash . . . 72

GlobalTransition . . . 73
HistoryDbg . . . 74
HistoryEntry . . . 76
IterativeVerif . . . 77
LocalModels . . . 77
LocalState . . . 78
LocalStateTemplate . . . 79
LocalTransition . . . 80
ModelParser . . . 81
ProbabilityEntry . . . 82
ProbabilityStrategyDecisions . . . 83
ProbabilityTrueFalse . . . 83
ProbNode . . . 89
- ProbAdd . . . 83
- ProbConst . . . 85
- ProbDiv . . . 86
- ProbMul . . . 87
- ProbSub . . . 90

Result . . . 91
StateVerificationInfo . . . 91
StrategyBitsComparator . . . 92
StrategyCollection . . . 92
StrategyCollectionTemplate . . . 93
StrategyEntry . . . 93
StrategyParser . . . 94
TestVerif . . . 95
TransitionTemplate . . . 95
Var . . . 96
Verification . . . 97
VerificationIterative . . . 110
yy_buffer_state . . . 120
yy_trans_info . . . 121
yyalloc . . . 121
YYSTYPE . . . 121

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Action
Single action template for probabilistic verification 15
ActionTemplate
Represents a selected action 15
Agent
Contains all data for a single Agent, including id, name and all of the agents' variables 16
AgentTemplate
Represents a single agent loaded from the description from a file 18
Assignment
Represents an assingment 23
Cfg 24
Condition
Represents a condition for LocalTransition 25
DecisionEntry 25
DotGraph 26
EpistemicClass
Represents a single epistemic class 28
ExprAdd
Node for addition 29
ExprAnd
Node for AND operator 31
ExprConst
Node for a constant 32
ExprDiv
Node for division 34
ExprEq
Node for "==" operator 36
ExprGe
Node for ">=" operator 37
ExprGt
Node for ">" operator 39
ExprHart
Node for a Hartley operator 41
ExprIdent
Node for an identifier 42

ExprKnow
Node for a knowledge operator . . . 45
ExprLe
Node for "<=" operator . . . 46
ExprLt
Node for "<" operator . . . 48
ExprMul
Node for multiplication . . . 50
ExprNe
Node for "!=" operator . . . 51
ExprNode
Base node for expressions . . . 53
ExprNot
Node for NOT operator . . . 54
ExprOr
Node for OR operator . . . 55
ExprRem
Node for modulo . . . 56
ExprSub
Node for subtraction . . . 58
Formula
Contains the processed verification formula . . . 60
FormulaTemplate
Contains a template for coalition of Agent as string from the formula . . . 60
GlobalModel
Represents a global model, containing agents and a formula . . . 61
GlobalModelGenerator
Stores the local models, formula and a global model . . . 62
GlobalState
Represents a single global state . . . 71
GlobalModelGenerator::GlobalStateTupleHash . . . 72
GlobalTransition
Represents a single global transition . . . 73
HistoryDbg
Stores history and allows displaying it to the console . . . 74
HistoryEntry
Structure used to save model traversal history . . . 76
IterativeVerif . . . 77
LocalModels
Represents a single local model, contains all agents and variables . . . 77
LocalState
Represents a single LocalState, containing id, name and internal variables . . . 78
LocalStateTemplate
A template for the local state . . . 79
LocalTransition
Represents a single local transition, containing id, global name, local name, is shared and count of the appearances . . . 80
ModelParser
A parser for converting a text file into a model . . . 81
ProbabilityEntry
Class for handling the probability operations during probability verification . . . 82
ProbabilityStrategyDecisions
Structure for the iterative set strategy generation . . . 83
ProbabilityTrueFalse
Contains probability of a state being in a true or false state . . . 83
ProbAdd
Node for addition . . . 83

ProbConst
Node for a constant . . . 85
ProbDiv
Node for division . . . 86
ProbMul
Node for multiplication . . . 87
ProbNode
Base node for probability calculations . . . 89
ProbSub
Node for subtraction . . . 90
Result . . . 91
StateVerificationInfo
Handles all single state verification information during probabilistic verification . . . 91
StrategyBitsComparator
A comparator for two bitsets containing values for the global states . . . 92
StrategyCollection
Handles currently selected strategy when the strategy is set . . . 92
StrategyCollectionTemplate . . . 93
StrategyEntry . . . 93
StrategyParser
A parser for converting a text file into a strategy . . . 94
TestVerif . . . 95
TransitionTemplate
Represents a meta-transition . . . 95
Var
Represents a variable in the model, containing name, initial value and persistence . . . 96
Verification
A class that verifies if the model fulfills the formula. Also can do some operations on decision history . . . 97
VerificationIterative
A class that verifies if the model fulfills the formula. Also can do some operations on decision history . . . 110
yy_buffer_state . . . 120
yy_trans_info . . . 121
yyalloc . . . 121
YYSTYPE . . . 121

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

Agent.cpp
Class of an agent. Class of an agent 123
Agent.hpp
Class of an agent. Class of an agent 123
clock_test.h . . . . . **??**
Common.hpp
Contains all commonly used classes and structs. Contains all commonly used classes and structs 124
ConditionOperator.hpp . . . . . **??**
config.h . . . . . **??**
Constants.hpp . . . . . **??**
ctl_test.h . . . . . **??**
DotGraph.cpp
Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax 124
DotGraph.hpp
Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax 124
EpistemicClass.hpp
Struct of an epistemic class. Struct of an epistemic class 125
expressions.cc
Eval and helper class for expressions. Eval and helper class for expressions 126
expressions.hpp
Eval and helper class for expressions. Eval and helper class for expressions 126
general_test.h . . . . . **??**
GlobalModel.hpp
Struct of a global model. Struct of a global model 127
GlobalModelGenerator.cpp
Generator of a global model. Class for initializing and generating a global model 128
GlobalModelGenerator.hpp
Generator of a global model. Class for initializing and generating a global model 128
GlobalState.cpp
Struct representing a global state. Struct representing a global state 129
GlobalState.hpp
Struct representing a global state. Struct representing a global state 129

GlobalStateVerificationStatus.hpp . . . . . ??
GlobalTransition.cpp
Struct representing a global transition. Struct representing a global transition 129
GlobalTransition.hpp
Struct representing a global transition. Struct representing a global transition 130
hartley_test.h . . . . . ??
input_error_test.h . . . . . ??
knowledge_test.h . . . . . ??
LocalState.cpp
Class representing a local state. Class representing a local state 130
LocalState.hpp
Class representing a local state. Class representing a local state 130
LocalTransition.hpp
Class representing a local transition. Class representing a local transition 131
ModelParser.cc
A model parser. A parser for converting a text file into a model 131
ModelParser.hpp . . . . . ??
natural_strategy_test.h . . . . . ??
new_clock_test.h . . . . . ??
new_simple_voting_run_test.h . . . . . ??
new_simple_voting_run_test_but_faster.h . . . . . ??
new_simple_voting_test.h . . . . . ??
new_simple_voting_test_but_faster.h . . . . . ??
new_simple_voting_with_fakes_test.h . . . . . ??
nodes.cc
Parser templates. Class for setting up a new objects from a parser 132
nodes.hpp
Parser templates. Class for setting up a new objects from a parser 132
parser.h . . . . . ??
probability_test.h . . . . . ??
recursion_test.h . . . . . ??
reduction_test.h . . . . . ??
simple_voting_run_test.h . . . . . ??
simple_voting_run_test_but_faster.h . . . . . ??
simple_voting_test.h . . . . . ??
simple_voting_test_but_faster.h . . . . . ??
simple_voting_with_fakes_test.h . . . . . ??
strategyNodes.hpp
Strategy parser templates. Class for setting up a new objects from a parser 133
StrategyParser.cc
A strategy file parser. A parser for converting a text file into a strategy 133
strategyParser.h . . . . . ??
StrategyParser.hpp
Strategy file parser. A parser for converting a text file into a strategy 134
test_test.h . . . . . ??
trains_test.h . . . . . ??
trains_with_bridge_test.h . . . . . ??
tree_test.h . . . . . ??
Types.hpp
Custom data structures. Data structures and classes containing model data 134
TypesDependency.hpp
Custom data structures, using other custom data structures. Data structures and classes containing model data that are using other custom data structures 135
Utils.cpp
Utility functions. A collection of utility functions to use in the project 137
Utils.hpp 140

Verification.cpp
Class for verification of the formula on a model. Class for verification of the specified formula on a specified model 143
Verification.hpp 144
VerificationIterative.cpp
Class for iteratively generated verification of the formula on a model. Class for iteratively generated verification of the specified formula on a specified model 146
VerificationIterative.hpp 147

Chapter 5

Class Documentation

5.1 Action Struct Reference

Single action template for probabilistic verification.

```
#include <TypesDependency.hpp>
```

Public Attributes

- vector< string > * **states**
- string **hash**
- string **actionName**

5.1.1 Detailed Description

Single action template for probabilistic verification.

The documentation for this struct was generated from the following file:

- TypesDependency.hpp

5.2 ActionTemplate Class Reference

Represents a selected action.

```
#include <strategyNodes.hpp>
```

Public Member Functions

- ActionTemplate (vector< string > *_states, string _actionName)

  Constructor for the ActionTemplate.

Public Attributes

- vector< string > * states

  Vector of states from which the action has to be executed from.
- string hash

  Contains the states in a.
- string **actionName**

5.2.1 Detailed Description

Represents a selected action.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 ActionTemplate()

```
ActionTemplate::ActionTemplate (
            vector< string > * _states,
            string _actionName )
```

Constructor for the ActionTemplate.

Constructor for an ActionTemplate.

Parameters

_states	Vector of states from which the action has to be executed from.
_actionName	Action executed from the aforementioned states.

The documentation for this class was generated from the following files:

- strategyNodes.hpp
- strategyNodes.cc

5.3 Agent Class Reference

Contains all data for a single Agent, including id, name and all of the agents' variables.

```
#include <Agent.hpp>
```

Public Member Functions

- Agent (int _id, string _name)

  Constructor for the Agent class, assigning it an id and name.
- LocalState * includesState (LocalState *state)

  Checks if there is an equivalent LocalState in the model to the one passed as an argment.

Public Attributes

- int id

  Identifier of the agent.
- string name

  Name of the agent.
- set< Var * > vars

  Variable names for the agent.
- LocalState * initState

  Initial state of the agent.
- vector< LocalState * > localStates

  Local states for this agent.
- vector< LocalTransition * > localTransitions

  Local transitions for this agent.

5.3.1 Detailed Description

Contains all data for a single Agent, including id, name and all of the agents' variables.

5.3.2 Constructor & Destructor Documentation

5.3.2.1 Agent()

```
Agent::Agent (
            int _id,
            string _name )  [inline]
```

Constructor for the Agent class, assigning it an id and name.

Parameters

_id	Identifier of the new agent.
_name	Name of the new agent.

5.3.3 Member Function Documentation

5.3.3.1 includesState()

```
LocalState * Agent::includesState (
            LocalState * state )
```

Checks if there is an equivalent LocalState in the model to the one passed as an argment.

Parameters

state	A pointer to LocalState to be checked.

Returns

Returns a pointer to an equivalent LocalState if such exists, otherwise returns NULL.

The documentation for this class was generated from the following files:

- Agent.hpp
- Agent.cpp

5.4 AgentTemplate Class Reference

Represents a single agent loaded from the description from a file.

```
#include <nodes.hpp>
```

Public Member Functions

- AgentTemplate ()

  Constructor for an AgentTemplate.
- virtual AgentTemplate & setIdent (string _ident)

  Set the identifier of an agent.
- virtual AgentTemplate & setInitState (string _startState)

  Set the initial state of the agent.
- virtual AgentTemplate & addLocal (set< string > *variables)

  Adds local variables to an agent.
- virtual AgentTemplate & addPersistent (set< string > *variables)

  Adds persistent variables to an agent.
- virtual AgentTemplate & addInitial (set< Assignment * > *assigns)

  Adds initial assignments.
- virtual AgentTemplate & addTransition (TransitionTemplate *_transition)

  Adds a transition to the agent.
- virtual Agent * generateAgent (int id)

  Generate a new agent for the model.

Friends

- class **DotGraph**

5.4.1 Detailed Description

Represents a single agent loaded from the description from a file.

5.4.2 Member Function Documentation

5.4.2.1 addInitial()

```
AgentTemplate & AgentTemplate::addInitial (
            set< Assignment * > * assigns )  [virtual]
```

Adds initial assignments.

Sets initial values of agent's variables.

Parameters

assigns	Assignments to be added.

Returns

Returns a pointer to self.

Parameters

assigns	Set of variables to assign.

Returns

Returns itself.

5.4.2.2 addLocal()

```
AgentTemplate & AgentTemplate::addLocal (
            set< string > * variables )  [virtual]
```

Adds local variables to an agent.

Adds local variables to the agent.

Parameters

variables	Set of variables to be added.

Returns

Returns a pointer to self.

Parameters

variables	A pointer to a set of strings with the variables to be added.

Returns

Returns itself.

5.4.2.3 addPersistent()

```
AgentTemplate & AgentTemplate::addPersistent (
            set< string > * variables )  [virtual]
```

Adds persistent variables to an agent.

Adds persistent variables to the agent.

Parameters

variables	Set of variables to be added.

Returns

Returns a pointer to self.

Parameters

variables	A pointer to a set of strings with the variables to be added.

Returns

Returns itself.

5.4.2.4 addTransition()

```
AgentTemplate & AgentTemplate::addTransition (
            TransitionTemplate * _transition )  [virtual]
```

Adds a transition to the agent.

Adds a transition for the agent.

Parameters

_transition	Transition to be added.

Returns

Returns a pointer to self.

Parameters

_transition	Transition to be added.

Returns

Returns itself.

5.4.2.5 generateAgent()

```
Agent * AgentTemplate::generateAgent (
            int id )  [virtual]
```

Generate a new agent for the model.

Generates an agent for the model.

Parameters

id	Identification number defining a new Agent.

Returns

Returns a pointer to a new Agent.

Parameters

id	Identifier of the new Agent.

Returns

Returns a pointer to a newly created Agent.

5.4.2.6 setIdent()

```
AgentTemplate & AgentTemplate::setIdent (
            string _ident )  [virtual]
```

Set the identifier of an agent.

Sets the identifier of an agent.

Parameters

_ident	New agent identifier.

Returns

Returns a pointer to self.

Parameters

_ident	String with a new identifier.

Returns

Returns itself.

5.4.2.7 setInitState()

```
AgentTemplate & AgentTemplate::setInitState (
            string _initState )  [virtual]
```

Set the initial state of the agent.

Sets initial state of an agent.

Parameters

_startState	New inital agent state.

Returns

Returns a pointer to self.

Parameters

_initState	String with a new state.

Returns

Returns itself.

The documentation for this class was generated from the following files:

- nodes.hpp
- nodes.cc

5.5 Assignment Class Reference

Represents an assingment.

```
#include <nodes.hpp>
```

Public Member Functions

- Assignment (string _ident, ExprNode *_exp)

  Constructor for an Assignment class.
- virtual void assign (Environment &env)

  Make an assignment in a given environment.

Public Attributes

- string ident

  To what we should assign a value.
- ExprNode * value

  A value to be assigned.

5.5.1 Detailed Description

Represents an assingment.

5.5.2 Constructor & Destructor Documentation

5.5.2.1 Assignment()

```
Assignment::Assignment (
            string _ident,
            ExprNode * _exp )  [inline]
```

Constructor for an Assignment class.

Parameters

_ident	To what we should assign a value.
_exp	A value to be assigned.

5.5.3 Member Function Documentation

5.5.3.1 assign()

```
virtual void Assignment::assign (
            Environment & env )  [inline], [virtual]
```

Make an assignment in a given environment.

Parameters

env	Environment in which to make an assignment.

The documentation for this class was generated from the following file:

- nodes.hpp

5.6 Cfg Struct Reference

Public Attributes

- std::string fname

  path to input file with system specification
- int stv_mode

  stv_code as sum/combination of (1 - expandAllStates, 2 - verify, 4 - print metadata, 8 - run experiments)
- bool output_local_models

  (obsolete) print data on local model
- bool output_global_model

  (obsolete) print data on local model
- bool output_dot_files

  flag for .dot export (by default exports templates and local/global models)
- std::string dotdir

  pathprefix for .dot files export
- int **model_id**
- bool add_epsilon_transitions

  add epsilon transitions to the states in the model when it's blocked for some reason
- bool **formula_from_parameter**
- std::string **formula**
- bool **counterexample**
- bool **reduce**
- bool **reduce_all**
- std::string **reduce_args**
- bool **fixpoint**
- bool **natural_strategy**
- bool **probability** = false
- bool **verify_strategy**
- std::string **strategy_file_path**

The documentation for this struct was generated from the following file:

- Common.hpp

5.7 Condition Struct Reference

Represents a condition for LocalTransition.

```
#include <Types.hpp>
```

Public Attributes

- Var * var

  Pointer to a variable.
- ConditionOperator conditionOperator

  Conditional operator for the variable.
- int comparedValue

  Condition value to be met.

5.7.1 Detailed Description

Represents a condition for LocalTransition.

The documentation for this struct was generated from the following file:

- Types.hpp

5.8 DecisionEntry Struct Reference

Public Member Functions

- string **toString** ()

Public Attributes

- HistoryEntryType type = HistoryEntryType::CONTEXT

  Type of the history record.
- GlobalState * globalState = nullptr

  Saved global state.
- GlobalTransition * decision = nullptr

  Selected transition.
- bool globalTransitionControlled = false

  Is the transition controlled by an agent in coalition.
- GlobalStateVerificationStatus prevStatus = GlobalStateVerificationStatus::UNVERIFIED

  Previous model verification state.
- GlobalStateVerificationStatus newStatus = GlobalStateVerificationStatus::UNVERIFIED

  Next model verification state.
- int depth = 0

  Iteration depth.
- StrategyEntry strategy = StrategyEntry()

Holds currently processed strategy for the current state.

- float stateProbabilityPrevious = 0.0

  Holds previous GlobalState probability.
- ProbabilityCalculationType previousProbabilityEntryType = ProbabilityCalculationType::NONE_PROBABILITY

  Holds previous probability answer calculation method.
- ProbabilityCalculationType activeProbabilityEntryType = ProbabilityCalculationType::SUM_PROBABILITY

  Holds currently used probability calculation method, used to rollback correct probability value.
- ProbabilityTrueFalse previousProbabilityAnswerValues

  Holds previous true/false probability.

The documentation for this struct was generated from the following file:

- VerificationIterative.hpp

5.9 DotGraph Class Reference

Public Member Functions

- DotGraph ()

  empty graph
- DotGraph (GlobalModel ∗const gm, bool extended=false, bool correct=false)

  parses the transitions/states templates into nodes/edges
- DotGraph (Agent ∗const ag, bool extended=false)

  parses the local transitions/states templates into nodes/edges
- DotGraph (AgentTemplate ∗const at)

  parses the edge/state templates into nodes/edges
- void saveToFile (std::string pathprefix, std::string nameprefix, std::string basename="")

  creates a `.dot` file

Public Attributes

- std::vector< std::string > **nodes**
- std::vector< std::string > **edges**
- std::string **graphName**
- DotGraphBase **graphBase**

Static Public Attributes

- static string styleString

  (temporary hard-coded) graphviz visual configuration

Protected Member Functions

- void addNode (std::string id, std::string name)

  creates a node string in graphviz syntax
- void addEdge (std::string src, std::string trg, std::string label)

  creates an edge string in graphviz syntax

5.9.1 Member Function Documentation

5.9.1.1 addEdge()

```
void DotGraph::addEdge (
            std::string src,
            std::string trg,
            std::string label )  [protected]
```

creates an edge string in graphviz syntax

Parameters

src	id of a source node
trg	id of a target node
label	edge label and, possibly, extra attributes

5.9.1.2 addNode()

```
void DotGraph::addNode (
            std::string id,
            std::string name )  [protected]
```

creates a node string in graphviz syntax

Parameters

id	unique internal node identifier
name	displayed node label/name

5.9.1.3 saveToFile()

```
void DotGraph::saveToFile (
            std::string pathprefix,
            std::string nameprefix,
            std::string basename = "" )
```

creates a `.dot` file

Parameters

basename	name of file (parent graph name if blank)

5.9.2 Member Data Documentation

5.9.2.1 styleString

```
std::string DotGraph::styleString  [static]
```

Initial value:

```
=
    "\tedge[fontsize=\"10\"]\n"
    "\tnode [\n"
    "\t\tshape=circle,\n"

    "\t\twidth=auto,\n"
    "\t\tcolor=\"black\",\n"
    "\t\tfillcolor=\"#eeeeee\",\n"
    "\t\tstyle=\"filled,solid\",\n"
    "\t\tfontsize=8,\n"
    "\t\tfontname=\"Roboto\"\n"
    "\t]\n"
    "\tfontname=Consolas\n"
    "\tlayout=dot\n"
```

(temporary hard-coded) graphviz visual configuration

The documentation for this class was generated from the following files:

- DotGraph.hpp
- DotGraph.cpp

5.10 EpistemicClass Struct Reference

Represents a single epistemic class.

```
#include <EpistemicClass.hpp>
```

Public Attributes

- string hash

  Hash of that epistemic class.
- map< string, GlobalState * > globalStates

  Map of GlobalState hashes to according GlobalState pointers bound to this epistemic class.
- GlobalTransition * fixedCoalitionTransition

  Transition that was already selected in this epistemic class. Model has to choose this transition if it is already set.

5.10.1 Detailed Description

Represents a single epistemic class.

The documentation for this struct was generated from the following file:

- EpistemicClass.hpp

5.11 ExprAdd Class Reference

Node for addition.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprAdd (ExprNode *_larg, ExprNode *_rarg)

  Addition expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.11.1 Detailed Description

Node for addition.

5.11.2 Constructor & Destructor Documentation

5.11.2.1 ExprAdd()

```
ExprAdd::ExprAdd (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Addition expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.11.3 Member Function Documentation

5.11.3.1 eval()

```
int ExprAdd::eval (
            Environment & env,
```

```
GlobalModelGenerator * generator,
GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.12 ExprAnd Class Reference

Node for AND operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprAnd (ExprNode *_larg, ExprNode *_rarg)
  Logic AND expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.12.1 Detailed Description

Node for AND operator.

5.12.2 Constructor & Destructor Documentation

5.12.2.1 ExprAnd()

```
ExprAnd::ExprAnd (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Logic AND expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.12.3 Member Function Documentation

5.12.3.1 eval()

```
int ExprAnd::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.13 ExprConst Class Reference

Node for a constant.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprConst (int _val)

  Constant expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator ∗generator, GlobalState ∗globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.13.1 Detailed Description

Node for a constant.

5.13.2 Constructor & Destructor Documentation

5.13.2.1 ExprConst()

```
ExprConst::ExprConst (
            int _val )  [inline]
```

Constant expression constructor.

Parameters

_val	ExprConst value.

5.13.3 Member Function Documentation

5.13.3.1 eval()

```
int ExprConst::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.14 ExprDiv Class Reference

Node for division.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprDiv (ExprNode *_larg, ExprNode *_rarg)

  Division expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.14.1 Detailed Description

Node for division.

5.14.2 Constructor & Destructor Documentation

5.14.2.1 ExprDiv()

```
ExprDiv::ExprDiv (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Division expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.14.3 Member Function Documentation

5.14.3.1 eval()

```
int ExprDiv::eval (
            Environment & env,
```

```
GlobalModelGenerator * generator,
GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.15 ExprEq Class Reference

Node for "==" operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprEq (ExprNode *_larg, ExprNode *_rarg)

  Equals expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.15.1 Detailed Description

Node for "==" operator.

5.15.2 Constructor & Destructor Documentation

5.15.2.1 ExprEq()

```
ExprEq::ExprEq (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Equals expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.15.3 Member Function Documentation

5.15.3.1 eval()

```
int ExprEq::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.16 ExprGe Class Reference

Node for ">=" operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprGe (ExprNode *_larg, ExprNode *_rarg)

  Greater or equal expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.16.1 Detailed Description

Node for ">=" operator.

5.16.2 Constructor & Destructor Documentation

5.16.2.1 ExprGe()

```
ExprGe::ExprGe (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Greater or equal expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.16.3 Member Function Documentation

5.16.3.1 eval()

```
int ExprGe::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.17 ExprGt Class Reference

Node for ">" operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprGt (ExprNode *_larg, ExprNode *_rarg)

  Greater than expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.17.1 Detailed Description

Node for ">" operator.

5.17.2 Constructor & Destructor Documentation

5.17.2.1 ExprGt()

```
ExprGt::ExprGt (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Greater than expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.17.3 Member Function Documentation

5.17.3.1 eval()

```
int ExprGt::eval (
            Environment & env,
```

```
GlobalModelGenerator * generator,
GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.18 ExprHart Class Reference

Node for a Hartley operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprHart (string _agentName, bool _le, int _val, vector< ExprNode * > *_arg)

  Constant expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.18.1 Detailed Description

Node for a Hartley operator.

5.18.2 Constructor & Destructor Documentation

5.18.2.1 ExprHart()

```
ExprHart::ExprHart (
            string _agentName,
            bool _le,
            int _val,
            vector< ExprNode * > * _arg )  [inline]
```

Constant expression constructor.

Parameters

_agentName	Agent name for Hartley operator.
_le	Flag for if Hartley coefficient should be less equal or greater equal.
_val	Number to compare the result to.
_arg	Expression to verify with the given knowledge.

5.18.3 Member Function Documentation

5.18.3.1 eval()

```
int ExprHart::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.19 ExprIdent Class Reference

Node for an identifier.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprIdent (string _ident)

  Identifier expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int eval (Environment &env)

5.19.1 Detailed Description

Node for an identifier.

5.19.2 Constructor & Destructor Documentation

5.19.2.1 ExprIdent()

```
ExprIdent::ExprIdent (
            string _ident )  [inline]
```

Identifier expression constructor.

Parameters

_ident	ExprIdent value.

5.19.3 Member Function Documentation

5.19.3.1 eval() [1/2]

```
int ExprIdent::eval (
            Environment & env )  [virtual]
```

Parameters

env	

Returns

Implements ExprNode.

5.19.3.2 eval() [2/2]

```
int ExprIdent::eval (
            Environment & env,
```

```
GlobalModelGenerator * generator,
GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.20 ExprKnow Class Reference

Node for a knowledge operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprKnow (string _agentName, ExprNode *_arg)

  Constant expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.20.1 Detailed Description

Node for a knowledge operator.

5.20.2 Constructor & Destructor Documentation

5.20.2.1 ExprKnow()

```
ExprKnow::ExprKnow (
            string _agentName,
            ExprNode * _arg )  [inline]
```

Constant expression constructor.

Parameters

_agentName	Agent name for knowledge operator.
_arg	Expression to verify with the given knowledge.

5.20.3 Member Function Documentation

5.20.3.1 eval()

```
int ExprKnow::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.21 ExprLe Class Reference

Node for "<=" operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprLe (ExprNode *_larg, ExprNode *_rarg)

  Less or equal expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.21.1 Detailed Description

Node for "<=" operator.

5.21.2 Constructor & Destructor Documentation

5.21.2.1 ExprLe()

```
ExprLe::ExprLe (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Less or equal expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.21.3 Member Function Documentation

5.21.3.1 eval()

```
int ExprLe::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.22 ExprLt Class Reference

Node for "<" operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprLt (ExprNode *_larg, ExprNode *_rarg)

  Less than expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.22.1 Detailed Description

Node for "<" operator.

5.22.2 Constructor & Destructor Documentation

5.22.2.1 ExprLt()

```
ExprLt::ExprLt (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Less than expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.22.3 Member Function Documentation

5.22.3.1 eval()

```
int ExprLt::eval (
            Environment & env,
```

```
GlobalModelGenerator * generator,
GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.23 ExprMul Class Reference

Node for multiplication.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprMul (ExprNode *_larg, ExprNode *_rarg)

  Multiplication expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.23.1 Detailed Description

Node for multiplication.

5.23.2 Constructor & Destructor Documentation

5.23.2.1 ExprMul()

```
ExprMul::ExprMul (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Multiplication expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.23.3 Member Function Documentation

5.23.3.1 eval()

```
int ExprMul::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.24 ExprNe Class Reference

Node for "!=" operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprNe (ExprNode *_larg, ExprNode *_rarg)

  Not equals expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.24.1 Detailed Description

Node for "!=" operator.

5.24.2 Constructor & Destructor Documentation

5.24.2.1 ExprNe()

```
ExprNe::ExprNe (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Not equals expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.24.3 Member Function Documentation

5.24.3.1 eval()

```
int ExprNe::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.25 ExprNode Class Reference

Base node for expressions.

```
#include <expressions.hpp>
```

Public Member Functions

- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)=0

  Calculates the expression value.
- virtual int **eval** (Environment &env)=0

5.25.1 Detailed Description

Base node for expressions.

5.25.2 Member Function Documentation

5.25.2.1 eval()

```
virtual int ExprNode::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [pure virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implemented in ExprHart, ExprKnow, ExprGe, ExprGt, ExprLe, ExprLt, ExprNe, ExprEq, ExprNot, ExprOr, ExprAnd, ExprRem, ExprDiv, ExprMul, ExprSub, ExprAdd, ExprIdent, and ExprConst.

The documentation for this class was generated from the following file:

- expressions.hpp

5.26 ExprNot Class Reference

Node for NOT operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprNot (ExprNode *_arg)

  Logic NOT expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.26.1 Detailed Description

Node for NOT operator.

5.26.2 Constructor & Destructor Documentation

5.26.2.1 ExprNot()

```
ExprNot::ExprNot (
            ExprNode * _arg )  [inline]
```

Logic NOT expression constructor.

Parameters

_arg	Calculates the expression value.

5.26.3 Member Function Documentation

5.26.3.1 eval()

```
int ExprNot::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.27 ExprOr Class Reference

Node for OR operator.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprOr (ExprNode *_larg, ExprNode *_rarg)
  Logic OR expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)
  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.27.1 Detailed Description

Node for OR operator.

5.27.2 Constructor & Destructor Documentation

5.27.2.1 ExprOr()

```
ExprOr::ExprOr (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Logic OR expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.27.3 Member Function Documentation

5.27.3.1 eval()

```
int ExprOr::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.28 ExprRem Class Reference

Node for modulo.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprRem (ExprNode *_larg, ExprNode *_rarg)

  Modulo expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.28.1 Detailed Description

Node for modulo.

5.28.2 Constructor & Destructor Documentation

5.28.2.1 ExprRem()

```
ExprRem::ExprRem (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Modulo expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.28.3 Member Function Documentation

5.28.3.1 eval()

```
int ExprRem::eval (
            Environment & env,
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.29 ExprSub Class Reference

Node for subtraction.

```
#include <expressions.hpp>
```

Public Member Functions

- ExprSub (ExprNode *_larg, ExprNode *_rarg)

  Subtraction expression constructor.
- virtual int eval (Environment &env, GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual int **eval** (Environment &env)

5.29.1 Detailed Description

Node for subtraction.

5.29.2 Constructor & Destructor Documentation

5.29.2.1 ExprSub()

```
ExprSub::ExprSub (
            ExprNode * _larg,
            ExprNode * _rarg )  [inline]
```

Subtraction expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.29.3 Member Function Documentation

5.29.3.1 eval()

```
int ExprSub::eval (
            Environment & env,
```

```
GlobalModelGenerator * generator,
GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns an integer.

Implements ExprNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.30 Formula Struct Reference

Contains the processed verification formula.

```
#include <Types.hpp>
```

Public Attributes

- set< Agent * > coalition

  Coalition of Agent from the formula.
- vector< ExprNode * > * **p**
- bool **isF**
- bool **isCTL**
- float **probability** = 1
- ProbabilitySign **probabilitySign** = ProbabilitySign::NONE

5.30.1 Detailed Description

Contains the processed verification formula.

The documentation for this struct was generated from the following file:

- Types.hpp

5.31 FormulaTemplate Struct Reference

Contains a template for coalition of Agent as string from the formula.

```
#include <Types.hpp>
```

Public Attributes

- set< string > * **coalition**
- vector< ExprNode * > * **formula**
- bool **isF**
- ProbNode * **probability**
- string **probabilitySign** = ""

5.31.1 Detailed Description

Contains a template for coalition of Agent as string from the formula.

The documentation for this struct was generated from the following file:

- Types.hpp

5.32 GlobalModel Struct Reference

Represents a global model, containing agents and a formula.

```
#include <GlobalModel.hpp>
```

Public Attributes

- vector< Agent * > agents

  Pointers to all agents in a model.
- Formula * formula

  A pointer to a Formula.
- GlobalState * initState

  Pointer to the initial state of the model.
- vector< GlobalState * > globalStates

  Every GlobalState in the model.
- map< Agent *, map< string, EpistemicClass * > > epistemicClasses

  Map of Agent pointers to a map of EpistemicClass for graph traversal.
- map< Agent *, map< string, set< GlobalState * > > > epistemicClassesKnowledge

  Map of Agent pointers to a map of EpistemicClass for knowledge checks.

5.32.1 Detailed Description

Represents a global model, containing agents and a formula.

The documentation for this struct was generated from the following file:

- GlobalModel.hpp

5.33 GlobalModelGenerator Class Reference

Stores the local models, formula and a global model.

```
#include <GlobalModelGenerator.hpp>
```

Classes

- struct GlobalStateTupleHash

Public Member Functions

- GlobalModelGenerator ()

  Constructor for GlobalModelGenerator class.
- ~GlobalModelGenerator ()

  Destructor for GlobalModelGenerator class.
- GlobalState * initModel (LocalModels *localModels, Formula *formula)

  Initializes a global model from local models and a formula.
- void expandState (GlobalState *state)

  Goes through all GlobalTransition in a given GlobalState and creates new GlobalStates connected to the given one.
- vector< GlobalState * > expandStateAndReturn (GlobalState *state, bool returnAnyway=false)

  GlobalModelGenerator::expandState that also additionally returns a vector of newly created states.
- void expandAllStates ()

  Expands the states starting from the initial GlobalState and continues until there are no more states to expand.
- void expandAndReduceAllStates ()

  Expands and reduces the states starting from the initial GlobalState using a DFS-POR algorithm and continues until there are no more states to expand.
- GlobalModel * getCurrentGlobalModel ()

  Get for a GlobalModel used in initialization.
- Formula * getFormula ()

  Get for the Formula used in initialization.
- int getFormulaSize ()

  Get size of the Formula used in initialization.
- set< GlobalState * > * findOrCreateEpistemicClassForKnowledge (vector< LocalState * > *localStates, GlobalState *globalState, Agent *agent)

  Checks if a vector of LocalState is already an epistemic class for a given Agent, if not, creates a new one.
- Agent * getAgentInstanceByName (string agentName)

  Get a pointer to an agent by using its name.
- void markFormulaAsIncorrect ()

  Sets formula to an incorrectly written state.
- bool getFormulaCorectness ()

  Gets formula status.
- void initStrategy (StrategyCollection *strat)

  Initiates the strategy with a given StrategyCollection.
- void createIterativeStrategy (LocalModels *localModels)

  Prepares the strategy generating structures for the probabilistic verification.
- bool nextIterativeStrategy ()

  Generates next set strategy from the memorized coalition transitions for the probabilistic verification.
- string getCoalitionIdentifier (vector< LocalState * > *localStates)

  Returns a concatenated name of all given LocalStates.
- string getActionNameFromStateInStrategy (GlobalState *state)

  Gives next action's name from a given state.

Public Attributes

- map< Agent *, size_t > agentIndex

  auxiliary variable mapping Agent pointer to its index (replace size_t with if needed later)

Protected Member Functions

- GlobalState * generateInitState ()

  Generates initial state of the model from GlobalModel in memory.
- GlobalState * generateStateFromLocalStates (vector< LocalState * > *localStates, set< LocalTransition * > *viaLocalTransitions, GlobalState *prevGlobalState)

  Creates a new GlobalState using some of the internally known model data and given local states, transitions that were uset to get there and the previous global state.
- void generateGlobalTransitions (GlobalState *fromGlobalState, set< LocalTransition * > localTransitions, map< Agent *, vector< LocalTransition * >> transitionsByAgent)

  Adds all shared global transitions to a GlobalState.
- string computeEpistemicClassHash (vector< LocalState * > *localStates, Agent *agent)

  Creates a hash from a set of LocalState and an Agent.
- string computeGlobalStateHash (vector< LocalState * > *localStates)

  Creates a hash from a set of LocalState.
- EpistemicClass * findOrCreateEpistemicClass (vector< LocalState * > *localStates, Agent *agent)

  Checks if a vector of LocalState is already an epistemic class for a given Agent, if not, creates a new one.
- GlobalState * findGlobalStateInEpistemicClass (vector< LocalState * > *localStates, EpistemicClass *epistemicClass)

  Gets a GlobalState from an EpistemicClass if it exists in that episcemic class.

Protected Attributes

- LocalModels * localModels

  LocalModels used in initModel.
- Formula * formula

  Formula used in initModel.
- GlobalModel * globalModel

  GlobalModel created in initModel.
- bool correctModel

  Flag holding info if model is actually correct.
- unordered_map< GlobalState *, unordered_set< int > > candidateStateDepths

  Lookup map containing global states and the depths that the global state is contained in. Used for reductions.
- stack< GlobalState * > globalModelCandidates

  Candidate states to be used in reductions. Used for reductions.
- stack< int > stateDepths

  Saved depths of states added to statesToExpand. Used for reductions.
- unordered_set< GlobalState * > addedStates

  States that were added to a stack of states. Used for reductions.
- StrategyCollection * strategyCollection

  Strategy to check during verification (if any)
- ProbabilityStrategyDecisions **probabilityStrategy**

5.33.1 Detailed Description

Stores the local models, formula and a global model.

5.33.2 Member Function Documentation

5.33.2.1 computeEpistemicClassHash()

```
string GlobalModelGenerator::computeEpistemicClassHash (
            vector< LocalState * > * localStates,
            Agent * agent )  [protected]
```

Creates a hash from a set of LocalState and an Agent.

Parameters

localStates	Pointer to a vector of pointers of LocalState and pointer to and Agent to turn into a hash.

Returns

Returns a string with a hash.

5.33.2.2 computeGlobalStateHash()

```
string GlobalModelGenerator::computeGlobalStateHash (
            vector< LocalState * > * localStates )  [protected]
```

Creates a hash from a set of LocalState.

Parameters

localStates	Pointer to a vector of pointers of LocalState to turn into a hash.

Returns

Returns a string with a hash.

5.33.2.3 createIterativeStrategy()

```
void GlobalModelGenerator::createIterativeStrategy (
            LocalModels * localModels )
```

Prepares the strategy generating structures for the probabilistic verification.

Parameters

localModels	LocalModels to generate the coalition transition buckets from.

5.33.2.4 expandAndReduceAllStates()

```
void GlobalModelGenerator::expandAndReduceAllStates ( )
```

Expands and reduces the states starting from the initial GlobalState using a DFS-POR algorithm and continues until there are no more states to expand.

Parameters

depth	Current depth of the recursive generation of all states.

5.33.2.5 expandState()

```
void GlobalModelGenerator::expandState (
            GlobalState * state )
```

Goes through all GlobalTransition in a given GlobalState and creates new GlobalStates connected to the given one.

Parameters

state	A state from which the expansion should start.

5.33.2.6 expandStateAndReturn()

```
vector< GlobalState * > GlobalModelGenerator::expandStateAndReturn (
            GlobalState * state,
            bool returnAnyway = false )
```

GlobalModelGenerator::expandState that also additionally returns a vector of newly created states.

Parameters

state	A state from which the expansion should start.

5.33.2.7 findGlobalStateInEpistemicClass()

```
GlobalState * GlobalModelGenerator::findGlobalStateInEpistemicClass (
            vector< LocalState * > * localStates,
            EpistemicClass * epistemicClass )  [protected]
```

Gets a GlobalState from an EpistemicClass if it exists in that episcemic class.

Parameters

localStates	Pointer to a vector of pointers to LocalState, from which will be generated a global state hash.
epistemicClass	Epistemic class in which to check if a GlobalState exists.

Returns

Returns a pointer to a GlobalState if it exists in given epistemic class, otherwise returns nullptr.

5.33.2.8 findOrCreateEpistemicClass()

```
EpistemicClass * GlobalModelGenerator::findOrCreateEpistemicClass (
            vector< LocalState * > * localStates,
            Agent * agent )  [protected]
```

Checks if a vector of LocalState is already an epistemic class for a given Agent, if not, creates a new one.

Parameters

localStates	Local states from agent.
agent	Agent for which to check the existence of an epistemic class.

Returns

A pointer to a new or existing EpistemicClass.

5.33.2.9 findOrCreateEpistemicClassForKnowledge()

```
set< GlobalState * > * GlobalModelGenerator::findOrCreateEpistemicClassForKnowledge (
            vector< LocalState * > * localStates,
            GlobalState * global,
            Agent * agent )
```

Checks if a vector of LocalState is already an epistemic class for a given Agent, if not, creates a new one.

Parameters

localStates	Local states from agent.
agent	Agent for which to check the existence of an epistemic class.

Returns

A pointer to a new or existing EpistemicClass.

5.33.2.10 generateGlobalTransitions()

```
void GlobalModelGenerator::generateGlobalTransitions (
            GlobalState * fromGlobalState,
            set< LocalTransition * > localTransitions,
            map< Agent *, vector< LocalTransition * >> transitionsByAgent )  [protected]
```

Adds all shared global transitions to a GlobalState.

Parameters

fromGlobalState	Global state to add transitions to.
localTransitions	Initially empty, avaliable local transitions by each agent from transitionsByAgent.
transitionsByAgent	Mapped transitions to an agent, only with transitions avaliable for the agent at this moment.

5.33.2.11 generateInitState()

```
GlobalState * GlobalModelGenerator::generateInitState ( )  [protected]
```

Generates initial state of the model from GlobalModel in memory.

Returns

Returns a pointer to an initial GlobalState.

5.33.2.12 generateStateFromLocalStates()

```
GlobalState * GlobalModelGenerator::generateStateFromLocalStates (
            vector< LocalState * > * localStates,
            set< LocalTransition * > * viaLocalTransitions,
            GlobalState * prevGlobalState )  [protected]
```

Creates a new GlobalState using some of the internally known model data and given local states, transitions that were uset to get there and the previous global state.

Parameters

localStates	LocalStates from which the new GlobalState will be built.
viaLocalTransitions	Pointer to a set of pointers to LocalTransition from which the changes in variables, as a result of traversing through the transition, will be made in a new GlobalState.
prevGlobalState	Pointer to GlobalState from which all persistent variables will be copied over from to the new GlobalState.

Returns

Returns a pointer to a new or already existing in the same epistemic class GlobalModel.

5.33.2.13 getActionNameFromStateInStrategy()

```
string GlobalModelGenerator::getActionNameFromStateInStrategy (
            GlobalState * state )
```

Gives next action's name from a given state.

Parameters

state	GlobalState from which an action name would be retrieved if a set strategy is present.

Returns

String containing the action name ending on a semicolon.

5.33.2.14 getAgentInstanceByName()

```
Agent * GlobalModelGenerator::getAgentInstanceByName (
            string agentName )
```

Get a pointer to an agent by using its name.

Parameters

agentName	Name of an Agent that we want to get its instance.

Returns

Pointer to an Agent.

5.33.2.15 getCoalitionIdentifier()

```
string GlobalModelGenerator::getCoalitionIdentifier (
            vector< LocalState * > * localStates )
```

Returns a concatenated name of all given LocalStates.

Parameters

localStates	Vector of LocalStates to concatenate into a single string.

Returns

LocalState names separated with semicolons, ending on a semicolon.

5.33.2.16 getCurrentGlobalModel()

```
GlobalModel * GlobalModelGenerator::getCurrentGlobalModel ( )
```

Get for a GlobalModel used in initialization.

Returns

Returns a pointer to a global model.

5.33.2.17 getFormula()

```
Formula * GlobalModelGenerator::getFormula ( )
```

Get for the Formula used in initialization.

Returns

Returns a pointer to the formula structure.

5.33.2.18 getFormulaCorectness()

```
bool GlobalModelGenerator::getFormulaCorectness ( )
```

Gets formula status.

Returns

True if formula is written correctly, false otherwise.

5.33.2.19 getFormulaSize()

```
int GlobalModelGenerator::getFormulaSize ( )
```

Get size of the Formula used in initialization.

Returns

Returns the formula size.

5.33.2.20 initModel()

```
GlobalState * GlobalModelGenerator::initModel (
            LocalModels * localModels,
            Formula * formula )
```

Initializes a global model from local models and a formula.

Parameters

localModels	Pointer to LocalModels that will construct a global model.
formula	Pointer to a Formula to include into the model.

Returns

Returns a pointer to initial state of the global model.

5.33.2.21 initStrategy()

```
void GlobalModelGenerator::initStrategy (
            StrategyCollection * strat )
```

Initiates the strategy with a given StrategyCollection.

Parameters

strat	StrategyCollection to initialize the strategy.

5.33.2.22 nextIterativeStrategy()

```
bool GlobalModelGenerator::nextIterativeStrategy ( )
```

Generates next set strategy from the memorized coalition transitions for the probabilistic verification.

Returns

Returns true if the strategy was generated properly, false if it is the last permutation.

The documentation for this class was generated from the following files:

- GlobalModelGenerator.hpp
- GlobalModelGenerator.cpp

5.34 GlobalState Struct Reference

Represents a single global state.

```
#include <GlobalState.hpp>
```

Public Member Functions

- std::string toString (string indent="")

  Debug information on the given GlobalState.
- map< string, int > getGlobalStateEnvironment ()

  Get for the environment of a given global state.

Public Attributes

- string hash

  Hash of the global state used in quick checks if the states are in the same epistemic class.
- map< Agent *, EpistemicClass * > epistemicClasses

  Map of agents and the epistemic classes that belongs to the respective agent.
- bool isExpanded

  If false, the state can be still expanded, potentially creating new states, otherwise the expansion of the state already occured and is not necessary.
- GlobalStateVerificationStatus verificationStatus = GLOBAL_STATE_VERIFICATION_STATUS::UNVERIFIED

  Current verifivation status of this state.
- float probability = config.probability ? 0.0 : 1.0

  Collective probability of the state.
- set< GlobalTransition * > globalTransitions

  Every GlobalTransition in the model.
- set< GlobalState * > preimage

  Preimage for the global state, i.e. states from which there is a transition to this state.
- vector< LocalState * > localStatesProjection

  Local states of each agent that define this global state.
- map< Agent *, set< GlobalState * > * > epistemicClassesAllAgents

  States in the same epistemic class as the current one, for KBC.
- ProbabilityEntry **probabilityResult**
- GlobalTransition * probabilityLoop = nullptr

  Used to resolve self loops during probability calculation.
- VerifResult **stateVerifResult** = VerifResult::NOT_VERIFIED

5.34.1 Detailed Description

Represents a single global state.

5.34.2 Member Function Documentation

5.34.2.1 getGlobalStateEnvironment()

```
map< string, int > GlobalState::getGlobalStateEnvironment ( )
```

Get for the environment of a given global state.

Returns

A map of variable names and their values for the current global state.

5.34.2.2 toString()

```
std::string GlobalState::toString (
            string indent = "" )
```

Debug information on the given GlobalState.

Parameters

indent	- optional indentation string

Returns

GlobalState data

GlobalState data

The documentation for this struct was generated from the following files:

- GlobalState.hpp
- GlobalState.cpp

5.35 GlobalModelGenerator::GlobalStateTupleHash Struct Reference

Public Member Functions

- size_t **operator()** (const tuple< GlobalState *, int > &t) const

The documentation for this struct was generated from the following file:

- GlobalModelGenerator.hpp

5.36 GlobalTransition Struct Reference

Represents a single global transition.

```
#include <GlobalTransition.hpp>
```

Public Member Functions

- GlobalTransition ()

  Constructor for GlobalTransition.
- string joinLocalTransitionNames (char sep=';')

  Concatenates local transition names into a string.

Public Attributes

- uint32_t id

  Identifier of the transition.
- bool isInvalidDecision

  Marks if the transition is invalid, true if there is no point in traversing that transition, otherwise false.
- GlobalState * from

  Binding to a GlobalState from which this transition goes from.
- GlobalState * to

  Binding to a GlobalState from which this transition goes to.
- set< LocalTransition * > localTransitions

  Local transitions that define this global transition. A single transition or more in case of shared transitions.

Static Public Attributes

- static atomic_uint32_t **next_id**

5.36.1 Detailed Description

Represents a single global transition.

5.36.2 Member Function Documentation

5.36.2.1 joinLocalTransitionNames()

```
string GlobalTransition::joinLocalTransitionNames (
            char sep = ';' )
```

Concatenates local transition names into a string.

Parameters

sep	Separator used for separating the transition names.

Returns

String of transition names joined with the given separator and ending on the separator.

The documentation for this struct was generated from the following files:

- GlobalTransition.hpp
- GlobalTransition.cpp

5.37 HistoryDbg Class Reference

Stores history and allows displaying it to the console.

```
#include <Verification.hpp>
```

Public Member Functions

- HistoryDbg ()

  A constructor for HistoryDbg.
- ~HistoryDbg ()

  A destructor for HistoryDbg.
- void addEntry (HistoryEntry *entry)

  Adds a HistoryEntry to the debug history.
- void markEntry (HistoryEntry *entry, char chr)

  Marks an entry in the degug history with a char.
- void print (string prefix)

  Prints every entry from the algorithm's path.
- HistoryEntry * cloneEntry (HistoryEntry *entry)

  Checks if the HistoryEntry pointer exists in the debug history.

Public Attributes

- vector< pair< HistoryEntry *, char > > entries

  A pair of history entries and a char marking history type.

5.37.1 Detailed Description

Stores history and allows displaying it to the console.

5.37.2 Member Function Documentation

5.37.2.1 addEntry()

```
void HistoryDbg::addEntry (
            HistoryEntry * entry )
```

Adds a HistoryEntry to the debug history.

Parameters

entry	A pointer to the HistoryEntry that will be added to the history.

5.37.2.2 cloneEntry()

```
HistoryEntry * HistoryDbg::cloneEntry (
            HistoryEntry * entry )
```

Checks if the HistoryEntry pointer exists in the debug history.

Parameters

entry	A pointer to a HistoryEntry to be checked.

Returns

Identity function if the entry is in history, otherwise returns nullptr.

5.37.2.3 markEntry()

```
void HistoryDbg::markEntry (
            HistoryEntry * entry,
            char chr )
```

Marks an entry in the degug history with a char.

Parameters

entry	A pointer to a HistoryEntry that is supposed to be marked.
chr	A char that will be made into a pair with a HistoryEntry.

5.37.2.4 print()

```
void HistoryDbg::print (
            string prefix )
```

Prints every entry from the algorithm's path.

Parameters

prefix	A prefix string to append to the front of every entry.

The documentation for this class was generated from the following files:

- Verification.hpp
- Verification.cpp

5.38 HistoryEntry Struct Reference

Structure used to save model traversal history.

```
#include <Verification.hpp>
```

Public Member Functions

- string toString ()

  Converts HistoryEntry to string.

Public Attributes

- HistoryEntryType type

  Type of the history record.
- GlobalState * globalState

  Saved global state.
- GlobalTransition * decision

  Selected transition.
- bool globalTransitionControlled

  Is the transition controlled by an agent in coalition.
- GlobalStateVerificationStatus prevStatus

  Previous model verification state.
- GlobalStateVerificationStatus newStatus

  Next model verification state.
- int depth

  Recursion depth.
- StrategyEntry strategy

  Holds currently processed strategy for the current state.
- HistoryEntry * prev

  Pointer to the previous HistoryEntry.
- HistoryEntry * next

  Pointer to the next HistoryEntry.

5.38.1 Detailed Description

Structure used to save model traversal history.

5.38.2 Member Function Documentation

5.38.2.1 toString()

```
string HistoryEntry::toString ( ) [inline]
```

Converts HistoryEntry to string.

Returns

A string with the descriprion of this history record.

The documentation for this struct was generated from the following file:

- Verification.hpp

5.39 IterativeVerif Class Reference

Public Member Functions

- **IterativeVerif** (string path)
- **IterativeVerif** (string path, int mode)
- bool **verify** (string path, GlobalModelGenerator *generator)

Public Attributes

- GlobalModelGenerator * **generator** = new GlobalModelGenerator()
- bool **result**
- string **knowledge**
- string **hartley**
- int **hCoeff**
- bool **le**

The documentation for this class was generated from the following file:

- config.h

5.40 LocalModels Struct Reference

Represents a single local model, contains all agents and variables.

```
#include <Types.hpp>
```

Public Attributes

- vector< Agent * > agents

  A vector of agents for the current model.

5.40.1 Detailed Description

Represents a single local model, contains all agents and variables.

The documentation for this struct was generated from the following file:

- Types.hpp

5.41 LocalState Class Reference

Represents a single LocalState, containing id, name and internal variables.

```
#include <LocalState.hpp>
```

Public Member Functions

- bool compare (LocalState *state)

  Function comparing two states.
- string toString (string indent="")

  Debug information on the given LocalState.

Public Attributes

- uint32_t id

  State identifier.
- string name

  State name.
- map< string, int > environment

  Local variables as a name and their current values.
- Agent * agent

  Binding to an Agent.
- set< LocalTransition * > localTransitions

  Binding to the set of LocalTransition.
- set< GlobalState * > epistemicGlobalStates

  Binding to the set of GlobalState that this LocalState belongs to.

5.41.1 Detailed Description

Represents a single LocalState, containing id, name and internal variables.

5.41.2 Member Function Documentation

5.41.2.1 compare()

```
bool LocalState::compare (
            LocalState * state )
```

Function comparing two states.

Parameters

state	A pointer to LocalState to which this state should be compared to.

Returns

Returns true if the current LocalState is the same as the passed one, otherwise false.

5.41.2.2 toString()

```
string LocalState::toString (
            string indent = "" )
```

Debug information on the given LocalState.

Parameters

indent	- optional indentation string

Returns

LocalState data

LocalState data

The documentation for this class was generated from the following files:

- LocalState.hpp
- LocalState.cpp

5.42 LocalStateTemplate Class Reference

A template for the local state.

```
#include <nodes.hpp>
```

Public Attributes

- string name

  Name of the local state.
- set< TransitionTemplate * > transitions

  Local transitions going out from this state.

5.42.1 Detailed Description

A template for the local state.

The documentation for this class was generated from the following file:

- nodes.hpp

5.43 LocalTransition Struct Reference

Represents a single local transition, containing id, global name, local name, is shared and count of the appearances.

```
#include <LocalTransition.hpp>
```

Public Attributes

- int id

  Identifier of the transition.
- string name

  Name of the transition (global).
- string localName

  Name of the transition (local).
- bool isShared

  Is the transition appearing somewhere else, true if yes, false if no.
- int sharedCount

  Count of recurring appearances of this transition.
- set< Condition * > conditions

  Conditions that have to be fulfilled for the transition to be avaliable.
- float probability

  Used for probability formula verification. The probability of this LocalTransition executing.
- Agent * agent

  Binding to an Agent.
- LocalState * from

  Binding to a LocalState from which this transition goes from.
- LocalState * to

  Binding to a LocalState from which this transition goes to.

5.43.1 Detailed Description

Represents a single local transition, containing id, global name, local name, is shared and count of the appearances.

The documentation for this struct was generated from the following file:

- LocalTransition.hpp

5.44 ModelParser Class Reference

A parser for converting a text file into a model.

```
#include <ModelParser.hpp>
```

Public Member Functions

- ModelParser ()

  ModelParser constructor.
- ~ModelParser ()

  ModelParser destructor.
- tuple< LocalModels, Formula > parse (string fileName)

  Parses a file with given name into a usable model.
- tuple< LocalModels, Formula > parseAndOverwriteFormula (string fileName, string s)

  Parses a text file in the given file path and internally replaces the verification formula with the given one.

5.44.1 Detailed Description

A parser for converting a text file into a model.

5.44.2 Member Function Documentation

5.44.2.1 parse()

```
tuple< LocalModels, Formula > ModelParser::parse (
            string fileName )
```

Parses a file with given name into a usable model.

Parameters

fileName	Name of the file to be converted into a model.

Returns

Pointer to a model created from a given file.

5.44.2.2 parseAndOverwriteFormula()

```
tuple< LocalModels, Formula > ModelParser::parseAndOverwriteFormula (
            string fileName,
            string s )
```

Parses a text file in the given file path and internally replaces the verification formula with the given one.

Parameters

fileName	Path to the text file.
s	New formula.

Returns

Returns a new model generation structure with a replaced formula.

The documentation for this class was generated from the following files:

- ModelParser.hpp
- ModelParser.cc

5.45 ProbabilityEntry Class Reference

Class for handling the probability operations during probability verification.

```
#include <TypesDependency.hpp>
```

Public Member Functions

- **ProbabilityEntry** (ProbabilityCalculationType mode)
- void **changeVerificationMode** (ProbabilityCalculationType mode)
- ProbabilityCalculationType **getVerificationMode** ()
- void **changeSumProbabilityTrue** (float changeProbabilityBy)
- void **changeSumProbabilityFalse** (float changeProbabilityBy)
- ProbabilityTrueFalse **getSumProbability** ()
- void **setSumProbability** (ProbabilityTrueFalse newSumProbability)
- void **changeMinProbabilityTrue** (float newProbability)
- void **changeMinProbabilityFalse** (float newProbability)
- ProbabilityTrueFalse **getMinProbability** ()
- void **setMinProbability** (ProbabilityTrueFalse newMinProbability)
- ProbabilityTrueFalse **returnProbability** ()

5.45.1 Detailed Description

Class for handling the probability operations during probability verification.

The documentation for this class was generated from the following file:

- TypesDependency.hpp

5.46 ProbabilityStrategyDecisions Class Reference

Structure for the iterative set strategy generation.

```
#include <TypesDependency.hpp>
```

Public Attributes

- map< Agent *, map< string, vector< StrategyCollection > > > **allCoalitionTransitions**
- map< string, int > **currentStrategyPermutation**

5.46.1 Detailed Description

Structure for the iterative set strategy generation.

The documentation for this class was generated from the following file:

- TypesDependency.hpp

5.47 ProbabilityTrueFalse Struct Reference

Contains probability of a state being in a true or false state.

```
#include <TypesDependency.hpp>
```

Public Attributes

- float **probabilityTrue** = 0.0
- float **probabilityFalse** = 0.0

5.47.1 Detailed Description

Contains probability of a state being in a true or false state.

The documentation for this struct was generated from the following file:

- TypesDependency.hpp

5.48 ProbAdd Class Reference

Node for addition.

```
#include <expressions.hpp>
```

Public Member Functions

- ProbAdd (ProbNode *_larg, ProbNode *_rarg)

  Addition expression constructor.
- virtual float eval (GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual float **eval** ()

5.48.1 Detailed Description

Node for addition.

5.48.2 Constructor & Destructor Documentation

5.48.2.1 ProbAdd()

```
ProbAdd::ProbAdd (
            ProbNode * _larg,
            ProbNode * _rarg )  [inline]
```

Addition expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.48.3 Member Function Documentation

5.48.3.1 eval()

```
float ProbAdd::eval (
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns a float.

Implements ProbNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.49 ProbConst Class Reference

Node for a constant.

```
#include <expressions.hpp>
```

Public Member Functions

- ProbConst (float _val)

  Constant expression constructor.
- virtual float eval (GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual float **eval** ()

5.49.1 Detailed Description

Node for a constant.

5.49.2 Constructor & Destructor Documentation

5.49.2.1 ProbConst()

```
ProbConst::ProbConst (
            float _val )  [inline]
```

Constant expression constructor.

Parameters

_val	ProbConst value.

5.49.3 Member Function Documentation

5.49.3.1 eval()

```
float ProbConst::eval (
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns a float.

Implements ProbNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.50 ProbDiv Class Reference

Node for division.

```
#include <expressions.hpp>
```

Public Member Functions

- ProbDiv (ProbNode *_larg, ProbNode *_rarg)

  Division expression constructor.
- virtual float eval (GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual float **eval** ()

5.50.1 Detailed Description

Node for division.

5.50.2 Constructor & Destructor Documentation

5.50.2.1 ProbDiv()

```
ProbDiv::ProbDiv (
            ProbNode * _larg,
            ProbNode * _rarg )  [inline]
```

Division expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.50.3 Member Function Documentation

5.50.3.1 eval()

```
float ProbDiv::eval (
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns a float.

Implements ProbNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.51 ProbMul Class Reference

Node for multiplication.

```
#include <expressions.hpp>
```

Public Member Functions

- ProbMul (ProbNode *_larg, ProbNode *_rarg)

  Multiplication expression constructor.
- virtual float eval (GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual float **eval** ()

5.51.1 Detailed Description

Node for multiplication.

5.51.2 Constructor & Destructor Documentation

5.51.2.1 ProbMul()

```
ProbMul::ProbMul (
            ProbNode * _larg,
            ProbNode * _rarg )  [inline]
```

Multiplication expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.51.3 Member Function Documentation

5.51.3.1 eval()

```
float ProbMul::eval (
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns a float.

Implements ProbNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.52 ProbNode Class Reference

Base node for probability calculations.

```
#include <expressions.hpp>
```

Public Member Functions

- virtual float eval (GlobalModelGenerator *generator, GlobalState *globalState)=0

  Calculates the expression value.
- virtual float **eval** ()=0

5.52.1 Detailed Description

Base node for probability calculations.

5.52.2 Member Function Documentation

5.52.2.1 eval()

```
virtual float ProbNode::eval (
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [pure virtual]
```

Calculates the expression value.

Returns

Returns a float.

Implemented in ProbDiv, ProbMul, ProbSub, ProbAdd, and ProbConst.

The documentation for this class was generated from the following file:

- expressions.hpp

5.53 ProbSub Class Reference

Node for subtraction.

```
#include <expressions.hpp>
```

Public Member Functions

- ProbSub (ProbNode *_larg, ProbNode *_rarg)

  Subtraction expression constructor.
- virtual float eval (GlobalModelGenerator *generator, GlobalState *globalState)

  Calculates the expression value.
- virtual float **eval** ()

5.53.1 Detailed Description

Node for subtraction.

5.53.2 Constructor & Destructor Documentation

5.53.2.1 ProbSub()

```
ProbSub::ProbSub (
            ProbNode * _larg,
            ProbNode * _rarg )  [inline]
```

Subtraction expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.53.3 Member Function Documentation

5.53.3.1 eval()

```
float ProbSub::eval (
            GlobalModelGenerator * generator,
            GlobalState * globalState )  [virtual]
```

Calculates the expression value.

Parameters

env	Environment values.

Returns

Returns a float.

Implements ProbNode.

The documentation for this class was generated from the following files:

- expressions.hpp
- expressions.cc

5.54 Result Struct Reference

Public Attributes

- bool **verificationResult** = false
- ProbabilityTrueFalse **probabilityResult**

The documentation for this struct was generated from the following file:

- Types.hpp

5.55 StateVerificationInfo Struct Reference

Handles all single state verification information during probabilistic verification.

```
#include <TypesDependency.hpp>
```

Public Attributes

- GlobalState ∗ **globalState** = nullptr
- StateVerificationInfo ∗ **fromState** = nullptr
- queue< GlobalTransition ∗ > **controlledTransitionsLeftToProcess**
- queue< GlobalTransition ∗ > **uncontrolledTransitionsLeftToProcess**
- map< string, set< GlobalTransition ∗ > > **controlledProbabilisticTransitionsLeftToProcess**
- map< string, set< GlobalTransition ∗ > > **uncontrolledProbabilisticTransitionsLeftToProcess**
- int **depth** = 0
- bool **processed** = false
- VerifResult **verifResult** = VerifResult::NOT_VERIFIED
- bool **controlled** = false
- bool **uncontrolled** = false
- bool **hasControlledProbabilistic** = false
- bool **hasUncontrolledProbabilistic** = false
- bool **hasValidControlledTransition** = false
- bool **hasValidUncontrolledTransition** = true
- bool **isControlledByCoalition** = false
- bool **gotThroughPreselectedTransition** = false
- bool **gotResponseFromOtherState** = false

5.55.1 Detailed Description

Handles all single state verification information during probabilistic verification.

The documentation for this struct was generated from the following file:

- TypesDependency.hpp

5.56 StrategyBitsComparator Struct Reference

A comparator for two bitsets containing values for the global states.

```
#include <TypesDependency.hpp>
```

Public Member Functions

- bool **operator()** (const bitset< STRATEGY_BITS > &b1, const bitset< STRATEGY_BITS > &b2) const

5.56.1 Detailed Description

A comparator for two bitsets containing values for the global states.

The documentation for this struct was generated from the following file:

- TypesDependency.hpp

5.57 StrategyCollection Class Reference

Handles currently selected strategy when the strategy is set.

```
#include <TypesDependency.hpp>
```

Public Member Functions

- **StrategyCollection** (string hash, Action action)
- void **addAction** (Action action)
- void **addStrategy** (StrategyCollection strategy)
- void **clear** ()
- map< string, Action > **getStrategy** ()

5.57.1 Detailed Description

Handles currently selected strategy when the strategy is set.

The documentation for this class was generated from the following file:

- TypesDependency.hpp

5.58 StrategyCollectionTemplate Class Reference

Public Member Functions

- void addAction (vector< string > *_states, string _actionName)

Public Attributes

- map< string, ActionTemplate > **selectedStrategy**

5.58.1 Member Function Documentation

5.58.1.1 addAction()

```
void StrategyCollectionTemplate::addAction (
            vector< string > * _states,
            string _actionName )
```

Parameters

newAction	

The documentation for this class was generated from the following files:

- strategyNodes.hpp
- strategyNodes.cc

5.59 StrategyEntry Struct Reference

Public Attributes

- StrategyEntryType **type** = NOT_MODIFIED
- bitset< STRATEGY_BITS > **globalValues** = 0
- string * **actionName**

The documentation for this struct was generated from the following file:

- TypesDependency.hpp

5.60 StrategyParser Class Reference

A parser for converting a text file into a strategy.

```
#include <StrategyParser.hpp>
```

Public Member Functions

- StrategyParser ()

  StrategyParser constructor.
- ~StrategyParser ()

  ModelParser destructor.
- StrategyCollection * parse (string fileName)

  Parses a file with given name into a usable strategy.

5.60.1 Detailed Description

A parser for converting a text file into a strategy.

5.60.2 Member Function Documentation

5.60.2.1 parse()

```
StrategyCollection * StrategyParser::parse (
            string fileName )
```

Parses a file with given name into a usable strategy.

Parameters

fileName	Name of the file to be converted into a strategy.

Returns

Pointer to a model created from a given file.

The documentation for this class was generated from the following files:

- StrategyParser.hpp
- StrategyParser.cc

5.61 TestVerif Class Reference

Public Member Functions

- **TestVerif** (string path)
- **TestVerif** (string path, int mode)
- bool **verify** (string path, GlobalModelGenerator ∗generator)
- bool **verifyFull** (string path, GlobalModelGenerator ∗generator)
- bool **reducedVerify** (string path, GlobalModelGenerator ∗generator)
- bool **strategyVerify** (string path, GlobalModelGenerator ∗generator)
- bool **probabilityVerify** (string path, GlobalModelGenerator ∗generator)

Public Attributes

- GlobalModelGenerator ∗ **generator** = new GlobalModelGenerator()
- bool **result**
- string **knowledge**
- string **hartley**
- int **hCoeff**
- bool **le**

The documentation for this class was generated from the following file:

- config.h

5.62 TransitionTemplate Class Reference

Represents a meta-transition.

`#include <nodes.hpp>`

Public Member Functions

- TransitionTemplate (int _shared, string _patternName, string _matchName, string _startState, string _end↩State, ExprNode ∗_cond, set< Assignment ∗ > ∗_assign, ProbNode ∗_prob)

  TransitionTemplate constructor.

Public Attributes

- int shared

  Needed amound of needed agents. -1 if not shared.
- string patternName

  Name of the pattern.
- string matchName

  Global name for shared transitions.
- string startState

  Start state name.
- string endState

  End state name.
- ExprNode ∗ condition

  Condition expression that has do be fulfilled in that transition.
- set< Assignment ∗ > ∗ assignments

  Set of assignments.
- ProbNode ∗ probability

  Probability of executing the action.

5.62.1 Detailed Description

Represents a meta-transition.

5.62.2 Constructor & Destructor Documentation

5.62.2.1 TransitionTemplate()

```
TransitionTemplate::TransitionTemplate (
            int _shared,
            string _patternName,
            string _matchName,
            string _startState,
            string _endState,
            ExprNode * _cond,
            set< Assignment * > * _assign,
            ProbNode * _prob )  [inline]
```

TransitionTemplate constructor.

Parameters

_shared	Needed amound of needed agents. -1 if not shared.
_patternName	Name of the pattern.
_matchName	Global name for shared transitions.
_startState	Start state name.
_endState	End state name.
_cond	Condition expression that has do be fulfilled in that transition.
_assign	Set of assignments.
_prob	Probability of executing the action.

The documentation for this class was generated from the following file:

- nodes.hpp

5.63 Var Struct Reference

Represents a variable in the model, containing name, initial value and persistence.

```
#include <Types.hpp>
```

Public Attributes

- string name

  Variable name.
- int initialValue

  Initial value of the variable.
- bool persistent

  True if variable is persistent, i.e. it should appear in all states in the model, false otherwise.
- Agent * agent

  Reference to an agent, to which this variable belongs to.

5.63.1 Detailed Description

Represents a variable in the model, containing name, initial value and persistence.

The documentation for this struct was generated from the following file:

- Types.hpp

5.64 Verification Class Reference

A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

```
#include <Verification.hpp>
```

Public Member Functions

- Verification (GlobalModelGenerator *generator)

  Constructor for Verification.
- ~Verification ()

  Destructor for Verification.
- bool verify ()

  Starts the process of formula verification on a model.
- bool **fixpointVerify** ()
- void historyDecisionsERR ()

  Prints out the first ERR path.
- map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > getNaturalStrategy ()

  Get natural strategy map.
- vector< tuple< vector< tuple< bool, string > >, string > > getReducedStrategy ()

  Takes natural strategy from memory and reduces it using reduceStrategy().
- int getStrategyComplexity ()

  Natural strategy complexity before reduction. Each variable, negation, and, or are treated as value 1.
- Result verifyStrategy ()

  Verifies a strategy with the previously given models and formula.

Protected Member Functions

- bool verifyLocalStates (vector< LocalState * > *localStates, GlobalState *globalState)

  Verifies a set of LocalState that a GlobalState is composed of with a hardcoded formula.
- bool verifyGlobalState (GlobalState *globalState, int depth)

  Recursively verifies GlobalState.
- bool isGlobalTransitionControlledByCoalition (GlobalTransition *globalTransition)

  Checks if any of the LocalTransition in a given GlobalTransition has an Agent in a coalition in the formula.
- bool isAgentInCoalition (Agent *agent)

  Checks if the Agent is in a coalition based on the formula in a GlobalModelGenerator.
- EpistemicClass * getEpistemicClassForGlobalState (GlobalState *globalState)

  Gets the EpistemicClass for the agent in passed GlobalState, i.e. transitions from indistinguishable state from certain other states for an agent to other states.
- bool areGlobalStatesInTheSameEpistemicClass (GlobalState *globalState1, GlobalState *globalState2)

  Compares two GlobalState and checks if their EpistemicClass is the same.
- void addHistoryDecision (GlobalState *globalState, GlobalTransition *decision)

  Creates a HistoryEntry of the type DECISION and puts it on top of the stack of the decision history.
- void addHistoryStateStatus (GlobalState *globalState, GlobalStateVerificationStatus prevStatus, Global↩ StateVerificationStatus newStatus)

  Creates a HistoryEntry of the type STATE_STATUS and puts it to the top of the decision history.
- bool addHistoryContext (GlobalState *globalState, int depth, GlobalTransition *decision, bool global↩ TransitionControlled)

  Creates a HistoryEntry of the type CONTEXT and puts it to the top of the decision history.
- void addHistoryMarkDecisionAsInvalid (GlobalState *globalState, GlobalTransition *decision)

  Creates a HistoryEntry of the type MARK_DECISION_AS_INVALID and puts it to the top of the decision history.
- void addHistoryUncontrolledDecision (GlobalState *globalState, GlobalTransition *decision)

  Creates a HistoryEntry of the type UNCONTROLLED_DECISION and puts it on top of the stack of the decision history.
- HistoryEntry * newHistoryMarkDecisionAsInvalid (GlobalState *globalState, GlobalTransition *decision)

  Creates a HistoryEntry of the type MARK_DECISION_AS_INVALID and returns it.
- bool revertLastDecision (int depth)

  Reverts GlobalState and history to the previous decision state.
- void undoLastHistoryEntry (bool freeMemory)

  Removes the top entry of the history stack.
- void undoHistoryUntil (HistoryEntry *historyEntry, bool inclusive, int depth)

  Rolls back the history entries up to the certain HistoryEntry.
- void printCurrentHistory (int depth)

  Prints current history to the console.
- bool equivalentGlobalTransitions (GlobalTransition *globalTransition1, GlobalTransition *globalTransition2)

  Checks if two global transitions are made up of the same local transitions.
- bool checkUncontrolledSet (set< GlobalTransition * > uncontrolledGlobalTransitions, GlobalState *global↩ State, int depth, bool hasOmittedTransitions, bool mixed=false)

  Verifies if each transition from a given state yields a correct result.
- bool verifyTransitionSets (set< GlobalTransition * > controlledGlobalTransitions, set< GlobalTransition * > uncontrolledGlobalTransitions, GlobalState *globalState, int depth, bool hasOmittedTransitions, bool is↩ FMode, bool mixed=false)

  Checks if given transition sets are able to fulfill the formula for its given epistemic class.
- bool restoreHistory (GlobalState *globalState, GlobalTransition *globalTransition, int depth, bool controlled)

  Restores the decisions made for a given global state and transition in current recursion depth.
- bool **minFixpointVerify** ()
- bool **maxFixpointVerify** ()
- bitset< STRATEGY_BITS > globalStateToValueBits (GlobalState *globalState)

Changes globalState variables to a bitset used in natural strategy synthesis.

- vector< tuple< vector< tuple< bool, string > >, string > > reduceStrategy (vector< tuple< vector< tuple< bool, string >>, string >> strategyEntries, short lockedColumn=0, bool upperHalf=false)

  Natural strategy reduction algorithm.
- void increaseProbability (GlobalState *currentState, GlobalTransition *decision)

  Increases next GlobalState probability reached through the decision by adding to it a multiplied currentState probability with the decision probability.
- void lowerProbability (GlobalState *currentStateFrom, GlobalState *currentStateTo, set< LocalTransition * > decision)

  Decreases previous GlobalState probability reached from the decision by subtracting from it a multiplied currentState probability with the decision probability.

Protected Attributes

- TraversalMode mode

  Current mode of model traversal.
- GlobalState * revertToGlobalState

  Global state to which revert will rollback to.
- stack< HistoryEntry * > historyToRestore

  A history of decisions to be rolled back.
- GlobalModelGenerator * generator

  Holds current model and formula.
- HistoryEntry * historyStart

  Pointer to the start of model traversal history.
- HistoryEntry * historyEnd

  Pointer to the end of model traversal history.
- map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > naturalStrategy

  A table of actions paired vith globalState internal variables state for natural strategy construction.
- short strategyVariableLimit

  Max strategy variables found.
- vector< string > variableNames

  Easily readable variable names for natural strategy generation.
- int reductionComplexityBefore

  Natural strategy complexity before reduction.

5.64.1 Detailed Description

A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

5.64.2 Constructor & Destructor Documentation

5.64.2.1 Verification()

```
Verification::Verification (
            GlobalModelGenerator * generator )
```

Constructor for Verification.

Parameters

generator	Pointer to GlobalModelGenerator

5.64.3 Member Function Documentation

5.64.3.1 addHistoryContext()

```
bool Verification::addHistoryContext (
            GlobalState * globalState,
            int depth,
            GlobalTransition * decision,
            bool globalTransitionControlled )  [protected]
```

Creates a HistoryEntry of the type CONTEXT and puts it to the top of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
depth	Depth of the recursion of the validation algorithm.
decision	Pointer to a transition GlobalTransition selected by the algorithm.
globalTransitionControlled	True if the GlobalTransition is in the set of global transitions controlled by a coalition and it is not a fixed global transition.

Returns

Returns true if the natural strategy is good for now.

5.64.3.2 addHistoryDecision()

```
void Verification::addHistoryDecision (
            GlobalState * globalState,
            GlobalTransition * decision )  [protected]
```

Creates a HistoryEntry of the type DECISION and puts it on top of the stack of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
decision	Pointer to a GlobalTransition that is to be recorded in the decision history.

5.64.3.3 addHistoryMarkDecisionAsInvalid()

```
void Verification::addHistoryMarkDecisionAsInvalid (
            GlobalState * globalState,
            GlobalTransition * decision )  [protected]
```

Creates a HistoryEntry of the type MARK_DECISION_AS_INVALID and puts it to the top of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
decision	Pointer to a transition GlobalTransition selected by the algorithm.

5.64.3.4 addHistoryStateStatus()

```
void Verification::addHistoryStateStatus (
            GlobalState * globalState,
            GlobalStateVerificationStatus prevStatus,
            GlobalStateVerificationStatus newStatus )  [protected]
```

Creates a HistoryEntry of the type STATE_STATUS and puts it to the top of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
prevStatus	Previous GlobalStateVerificationStatus to be logged.
newStatus	New GlobalStateVerificationStatus to be logged.

5.64.3.5 addHistoryUncontrolledDecision()

```
void Verification::addHistoryUncontrolledDecision (
            GlobalState * globalState,
            GlobalTransition * decision )  [protected]
```

Creates a HistoryEntry of the type UNCONTROLLED_DECISION and puts it on top of the stack of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
decision	Pointer to a GlobalTransition that is to be recorded in the decision history.

5.64.3.6 areGlobalStatesInTheSameEpistemicClass()

```
bool Verification::areGlobalStatesInTheSameEpistemicClass (
            GlobalState * globalState1,
            GlobalState * globalState2 )  [protected]
```

Compares two GlobalState and checks if their EpistemicClass is the same.

Parameters

globalState1	Pointer to the first GlobalState.
globalState2	Pointer to the second GlobalState.

Returns

Returns true if the EpistemicClass is the same for both of the GlobalState. Returns false if they are different or at least one of them has no EpistemicClass.

5.64.3.7 checkUncontrolledSet()

```
bool Verification::checkUncontrolledSet (
            set< GlobalTransition * > uncontrolledGlobalTransitions,
            GlobalState * globalState,
            int depth,
            bool hasOmittedTransitions,
            bool mixed = false )  [protected]
```

Verifies if each transition from a given state yields a correct result.

Parameters

uncontrolledGlobalTransitions	A set of global transitions to be checked.
globalState	Currently processed global state.
depth	Current recursion depth.
hasOmittedTransitions	Flag with the information about skipped unneeded transitions.

Returns

Returns true if every transition yields a correct result, false otherwise.

5.64.3.8 equivalentGlobalTransitions()

```
bool Verification::equivalentGlobalTransitions (
            GlobalTransition * globalTransition1,
            GlobalTransition * globalTransition2 )  [protected]
```

Checks if two global transitions are made up of the same local transitions.

Parameters

globalTransition1	First global transition to compare.
globalTransition2	Second global transition to compare.

Returns

True if the two global transitions have the same local transitions, false otherwise.

5.64.3.9 getEpistemicClassForGlobalState()

```
EpistemicClass * Verification::getEpistemicClassForGlobalState (
            GlobalState * globalState )  [protected]
```

Gets the EpistemicClass for the agent in passed GlobalState, i.e. transitions from indistinguishable state from certain other states for an agent to other states.

Parameters

globalState	Pointer to a GlobalState of the model.

Returns

Pointer to the EpistemicClass that a coalition of agents from the formula belong to. If there is no such EpistemicClass, returns false.

5.64.3.10 getNaturalStrategy()

```
map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > Verification::getNatural↩
Strategy ( )
```

Get natural strategy map.

Returns

Map of variable values and action names.

5.64.3.11 getReducedStrategy()

```
vector< tuple< vector< tuple< bool, string > >, string > > Verification::getReducedStrategy
( )
```

Takes natural strategy from memory and reduces it using reduceStrategy().

Returns

Vector of tuples <vector of tuples <variable value, variable name>, action name>.

5.64.3.12 getStrategyComplexity()

```
int Verification::getStrategyComplexity ( )
```

Natural strategy complexity before reduction. Each variable, negation, and, or are treated as value 1.

Returns

Total sum of the natural strategy complexity before reduction.

5.64.3.13 globalStateToValueBits()

```
bitset< STRATEGY_BITS > Verification::globalStateToValueBits (
            GlobalState * globalState )  [protected]
```

Changes globalState variables to a bitset used in natural strategy synthesis.

Parameters

globalState	State that we want to extractvariable values from.

Returns

Bitset of environment variable values.

5.64.3.14 increaseProbability()

```
void Verification::increaseProbability (
            GlobalState * currentState,
            GlobalTransition * decision )  [protected]
```

Increases next GlobalState probability reached through the decision by adding to it a multiplied currentState probability with the decision probability.

Parameters

currentState	From which GlobalState the action was executed.
decision	GlobalTransition containing all transitions in the executed action and the next reachable GlobalState.

5.64.3.15 isAgentInCoalition()

```
bool Verification::isAgentInCoalition (
```

```
            Agent * agent )  [protected]
```

Checks if the Agent is in a coalition based on the formula in a GlobalModelGenerator.

Parameters

agent	Pointer to an Agent that is to be checked.

Returns

Returns true if the Agent is in a coalition, otherwise returns false.

5.64.3.16 isGlobalTransitionControlledByCoalition()

```
bool Verification::isGlobalTransitionControlledByCoalition (
            GlobalTransition * globalTransition )  [protected]
```

Checks if any of the LocalTransition in a given GlobalTransition has an Agent in a coalition in the formula.

Parameters

globalTransition	Pointer to a GlobalTransition in a model.

Returns

Returns true if the Agent is in coalition in the formula, otherwise returns false.

5.64.3.17 lowerProbability()

```
void Verification::lowerProbability (
            GlobalState * currentStateFrom,
            GlobalState * currentStateTo,
            set< LocalTransition * > decision )  [protected]
```

Decreases previous GlobalState probability reached from the decision by subtracting from it a multiplied current↩State probability with the decision probability.

Parameters

currentStateFrom	From which GlobalState the action was executed.
currentStateTo	To which grobal state the action was executed.
decision	Set of LocalTransition containing all transitions in the executed action.

5.64.3.18 newHistoryMarkDecisionAsInvalid()

```
HistoryEntry * Verification::newHistoryMarkDecisionAsInvalid (
            GlobalState * globalState,
            GlobalTransition * decision )  [protected]
```

Creates a HistoryEntry of the type MARK_DECISION_AS_INVALID and returns it.

Parameters

globalState	Pointer to a GlobalState of the model.
decision	Pointer to a transition GlobalTransition selected by the algorithm.

Returns

Returns pointer to a new HistoryEntry.

5.64.3.19 printCurrentHistory()

```
void Verification::printCurrentHistory (
            int depth )  [protected]
```

Prints current history to the console.

Parameters

depth	Integer that will be multiplied by 4 and appended as a prefix to the optional debug log.

5.64.3.20 reduceStrategy()

```
vector< tuple< vector< tuple< bool, string > >, string > > Verification::reduceStrategy (
            vector< tuple< vector< tuple< bool, string >>, string >> strategyEntries,
            short lockedColumn = 0,
            bool upperHalf = false )  [protected]
```

Natural strategy reduction algorithm.

Parameters

strategyEntries	Vector of tuples <vector of tuples <variable value, variable name>, action name> containing not reduced natural strategy.
lockedColumn	Index of the furthest leftmost locked column.
upperHalf	Indicates if the first columns of the processed strategy entries can't be shuffled. True if they can't be shuffled, false otherwise.

Returns

Vector of tuples <vector of tuples <variable value, variable name>, action name>.

5.64.3.21 restoreHistory()

```
bool Verification::restoreHistory (
            GlobalState * globalState,
            GlobalTransition * globalTransition,
            int depth,
            bool controlled )  [protected]
```

Restores the decisions made for a given global state and transition in current recursion depth.

Parameters

globalState	Currently processed global state.
globalTransition	Previously selected global transition from the given state to mark as invalid.
depth	Current recursion depth.
controlled	Flag with the information about current type of transition. True if controlled, false if uncontrolled.

Returns

Returns true if current top of the history entires matches with

5.64.3.22 revertLastDecision()

```
bool Verification::revertLastDecision (
            int depth )  [protected]
```

Reverts GlobalState and history to the previous decision state.

Parameters

depth	Integer that will be multiplied by 4 and appended as a prefix to the optional debug log.

Returns

Returns true if rollback is successful, otherwise returns false.

5.64.3.23 undoHistoryUntil()

```
void Verification::undoHistoryUntil (
            HistoryEntry * historyEntry,
            bool inclusive,
            int depth )  [protected]
```

Rolls back the history entries up to the certain HistoryEntry.

Parameters

historyEntry	Pointer to a HistoryEntry that the history has to be rolled back to.
inclusive	True if the rollback has to remove the specified entry too.
depth	Integer that will be multiplied by 4 and appended as a prefix to the optional debug log.

5.64.3.24 undoLastHistoryEntry()

```
void Verification::undoLastHistoryEntry (
            bool freeMemory )  [protected]
```

Removes the top entry of the history stack.

Parameters

freeMemory	True if the entry has to be removed from memory.

5.64.3.25 verify()

```
bool Verification::verify ( )
```

Starts the process of formula verification on a model.

Returns

Returns true if the verification is PENDING or VERIFIED_OK, otherwise returns false.

5.64.3.26 verifyGlobalState()

```
bool Verification::verifyGlobalState (
            GlobalState * globalState,
            int depth )  [protected]
```

Recursively verifies GlobalState.

Parameters

globalState	Pointer to a GlobalState of the model.
depth	Current depth of the recursion.

Returns

Returns true if the verification is PENDING or VERIFIED_OK, otherwise returns false.

5.64.3.27 verifyLocalStates()

```
bool Verification::verifyLocalStates (
            vector< LocalState * > * localStates,
            GlobalState * globalState )  [protected]
```

Verifies a set of LocalState that a GlobalState is composed of with a hardcoded formula.

Parameters

localStates	A pointer to a set of pointers to LocalState.

Returns

Returns true if there is a LocalState with a specific set of values, fulfilling the criteria, otherwise returns false.

5.64.3.28 verifyStrategy()

```
Result Verification::verifyStrategy ( )
```

Verifies a strategy with the previously given models and formula.

Returns

Result containing data of the verification result.

5.64.3.29 verifyTransitionSets()

```
bool Verification::verifyTransitionSets (
            set< GlobalTransition * > controlledGlobalTransitions,
            set< GlobalTransition * > uncontrolledGlobalTransitions,
            GlobalState * globalState,
            int depth,
            bool hasOmittedTransitions,
            bool isFMode,
            bool mixedTransitions = false )  [protected]
```

Checks if given transition sets are able to fulfill the formula for its given epistemic class.

Parameters

controlledGlobalTransitions	Set of controlled transitions in the current global state.
uncontrolledGlobalTransitions	Set of uncontrolled transitions in the current global state.
globalState	Currently processed global state.
depth	Current recursion depth.
hasOmittedTransitions	Flag with the information about skipped unneeded transitions.

Returns

True if there is a correct choice for an agent to take, false otherwise.

The documentation for this class was generated from the following files:

- Verification.hpp
- Verification.cpp

5.65 VerificationIterative Class Reference

A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

```
#include <VerificationIterative.hpp>
```

Public Member Functions

- VerificationIterative (GlobalModelGenerator *generator)

  Constructor for Verification.
- ~VerificationIterative ()

  Destructor for Verification.
- map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > getNaturalStrategy ()

  Get natural strategy map.
- vector< tuple< vector< tuple< bool, string > >, string > > getReducedStrategy ()

  Takes natural strategy from memory and reduces it using reduceStrategy().
- int getStrategyComplexity ()

  Natural strategy complexity before reduction. Each variable, negation, and, or are treated as value 1.
- Result verify ()

  Verifies a strategy with the previously given models and formula.

Protected Member Functions

- bool verifyLocalStates (vector< LocalState * > *localStates, GlobalState *globalState)

  Verifies a set of LocalState that a GlobalState is composed of with a hardcoded formula.
- bool isGlobalTransitionControlledByCoalition (GlobalTransition *globalTransition)

  Checks if any of the LocalTransition in a given GlobalTransition has an Agent in a coalition in the formula.
- bool isAgentInCoalition (Agent *agent)

  Checks if the Agent is in a coalition based on the formula in a GlobalModelGenerator.
- EpistemicClass * getEpistemicClassForGlobalState (GlobalState *globalState)

  Gets the EpistemicClass for the agent in passed GlobalState, i.e. transitions from indistinguishable state from certain other states for an agent to other states.
- bool areGlobalStatesInTheSameEpistemicClass (GlobalState *globalState1, GlobalState *globalState2)

  Compares two GlobalState and checks if their EpistemicClass is the same.
- string checkIfProbabilistic (GlobalTransition *transitionToCheck)

  Checks if a given transition is a probabilistic one.
- void findLoopedProbabilityTransitions (map< string, set< GlobalTransition * >> *transitionSets)

  Looks for the probability transitions with loops and dissolves them.
- void dissolveLoopedProbabilityTransition (GlobalTransition *transitionToDissolve, set< GlobalTransition * > *probabilityTransitions)

  Marks a loop as a transition to dissolve.
- void addHistoryDecision (GlobalState *globalState, GlobalTransition *decision)

  Creates a HistoryEntry of the type DECISION and puts it on top of the stack of the decision history.
- void addHistoryStateStatus (GlobalState *globalState, GlobalStateVerificationStatus prevStatus, Global↩StateVerificationStatus newStatus)

  Creates a HistoryEntry of the type STATE_STATUS and puts it to the top of the decision history.
- bool addHistoryContext (GlobalState *globalState, int depth, GlobalTransition *decision, bool global↩TransitionControlled)

  Creates a HistoryEntry of the type CONTEXT and puts it to the top of the decision history.
- void **addHistoryProbability** (GlobalTransition *decision)
- void addHistoryAnswerProbability (StateVerificationInfo *globalState, ProbabilityCalculationType current↩Mode, ProbabilityTrueFalse newProbabilityValues)

  Adds a history entry for the change of probabilistic answer in a particular state.
- void addHistoryMarkDecisionAsInvalid (GlobalState *globalState, GlobalTransition *decision)

  Creates a HistoryEntry of the type MARK_DECISION_AS_INVALID and puts it to the top of the decision history.
- bool revertToLastDecision ()

  Reverts the taken actions to the paint of the last decision.
- void undoLastHistoryEntry ()

  Removes the top entry of the history stack.
- bool equivalentGlobalTransitions (GlobalTransition *globalTransition1, GlobalTransition *globalTransition2)

  Checks if two global transitions are made up of the same local transitions.
- bitset< STRATEGY_BITS > globalStateToValueBits (GlobalState *globalState)

  Changes globalState variables to a bitset used in natural strategy synthesis.
- vector< tuple< vector< tuple< bool, string > >, string > > reduceStrategy (vector< tuple< vector< tuple< bool, string >>, string >> strategyEntries, short lockedColumn=0, bool upperHalf=false)

  Natural strategy reduction algorithm.
- bool testForAndFixBadAgents (StateVerificationInfo *stateVerificationInfo)

  Fixes potentially blocking agent actions.

Protected Attributes

- GraphTraversalMode mode = GraphTraversalMode::NORMAL_TRAVERSAL

  Current mode of model traversal.
- GlobalState * revertToGlobalState

  Global state to which revert will rollback to.
- GlobalModelGenerator * generator

  Holds current model and formula.
- stack< DecisionEntry > history

  Stack of decisions made.
- map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > naturalStrategy

  A table of actions paired vith globalState internal variables state for natural strategy construction.
- short strategyVariableLimit

  Max strategy variables found.
- vector< string > variableNames

  Easily readable variable names for natural strategy generation.
- int reductionComplexityBefore

  Natural strategy complexity before reduction.
- stack< StateVerificationInfo > **statesToProcess**

5.65.1 Detailed Description

A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

5.65.2 Constructor & Destructor Documentation

5.65.2.1 VerificationIterative()

```
VerificationIterative::VerificationIterative (
            GlobalModelGenerator * generator )
```

Constructor for Verification.

Parameters

generator	Pointer to GlobalModelGenerator

5.65.3 Member Function Documentation

5.65.3.1 addHistoryAnswerProbability()

```
void VerificationIterative::addHistoryAnswerProbability (
            StateVerificationInfo * stateVerifInfo,
```

```
            ProbabilityCalculationType currentMode,
            ProbabilityTrueFalse newProbabilityValues )  [protected]
```

Adds a history entry for the change of probabilistic answer in a particular state.

Parameters

globalState	
currentMode	SUM_PROBABILITY if controlled, MIN_PROBABILITY if uncontrolled.
newProbabilityValues	

5.65.3.2 addHistoryContext()

```
bool VerificationIterative::addHistoryContext (
            GlobalState * globalState,
            int depth,
            GlobalTransition * decision,
            bool globalTransitionControlled )  [protected]
```

Creates a HistoryEntry of the type CONTEXT and puts it to the top of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
depth	Depth of the recursion of the validation algorithm.
decision	Pointer to a transition GlobalTransition selected by the algorithm.
globalTransitionControlled	True if the GlobalTransition is in the set of global transitions controlled by a coalition and it is not a fixed global transition.

Returns

Returns true if the natural strategy is good for now.

5.65.3.3 addHistoryDecision()

```
void VerificationIterative::addHistoryDecision (
            GlobalState * globalState,
            GlobalTransition * decision )  [protected]
```

Creates a HistoryEntry of the type DECISION and puts it on top of the stack of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
decision	Pointer to a GlobalTransition that is to be recorded in the decision history.

5.65.3.4 addHistoryMarkDecisionAsInvalid()

```
void VerificationIterative::addHistoryMarkDecisionAsInvalid (
            GlobalState * globalState,
            GlobalTransition * decision )  [protected]
```

Creates a HistoryEntry of the type MARK_DECISION_AS_INVALID and puts it to the top of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
decision	Pointer to a transition GlobalTransition selected by the algorithm.

5.65.3.5 addHistoryStateStatus()

```
void VerificationIterative::addHistoryStateStatus (
            GlobalState * globalState,
            GlobalStateVerificationStatus prevStatus,
            GlobalStateVerificationStatus newStatus )  [protected]
```

Creates a HistoryEntry of the type STATE_STATUS and puts it to the top of the decision history.

Parameters

globalState	Pointer to a GlobalState of the model.
prevStatus	Previous GlobalStateVerificationStatus to be logged.
newStatus	New GlobalStateVerificationStatus to be logged.

5.65.3.6 areGlobalStatesInTheSameEpistemicClass()

```
bool VerificationIterative::areGlobalStatesInTheSameEpistemicClass (
            GlobalState * globalState1,
            GlobalState * globalState2 )  [protected]
```

Compares two GlobalState and checks if their EpistemicClass is the same.

Parameters

globalState1	Pointer to the first GlobalState.
globalState2	Pointer to the second GlobalState.

Returns

Returns true if the EpistemicClass is the same for both of the GlobalState. Returns false if they are different or at least one of them has no EpistemicClass.

5.65.3.7 checkIfProbabilistic()

```
string VerificationIterative::checkIfProbabilistic (
            GlobalTransition * transitionToCheck )  [protected]
```

Checks if a given transition is a probabilistic one.

Parameters

transitionToCheck	Transition to be checked if posesses probability != 1.

Returns

Returns concatenated names of local transitions separated with semicolons if true, returns empty string otherwise.

5.65.3.8 dissolveLoopedProbabilityTransition()

```
void VerificationIterative::dissolveLoopedProbabilityTransition (
            GlobalTransition * transitionToDissolve,
            set< GlobalTransition * > * probabilityTransitions )  [protected]
```

Marks a loop as a transition to dissolve.

Parameters

transitionToDissolve	GlobalTransition to ignore.
probabilityTransitions	Set of GlobalTransitions to remove the transition from (unused)

5.65.3.9 equivalentGlobalTransitions()

```
bool VerificationIterative::equivalentGlobalTransitions (
            GlobalTransition * globalTransition1,
            GlobalTransition * globalTransition2 )  [protected]
```

Checks if two global transitions are made up of the same local transitions.

Parameters

globalTransition1	First global transition to compare.
globalTransition2	Second global transition to compare.

Returns

True if the two global transitions have the same local transitions, false otherwise.

5.65.3.10 findLoopedProbabilityTransitions()

```
void VerificationIterative::findLoopedProbabilityTransitions (
            map< string, set< GlobalTransition * >> * transitionSets )  [protected]
```

Looks for the probability transitions with loops and dissolves them.

Parameters

transitionSets	Set of GlobalTransitions to check for the loops.

5.65.3.11 getEpistemicClassForGlobalState()

```
EpistemicClass * VerificationIterative::getEpistemicClassForGlobalState (
            GlobalState * globalState )  [protected]
```

Gets the EpistemicClass for the agent in passed GlobalState, i.e. transitions from indistinguishable state from certain other states for an agent to other states.

Parameters

globalState	Pointer to a GlobalState of the model.

Returns

Pointer to the EpistemicClass that a coalition of agents from the formula belong to. If there is no such EpistemicClass, returns false.

5.65.3.12 getNaturalStrategy()

```
map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > VerificationIterative::get↩
NaturalStrategy ( )
```

Get natural strategy map.

Returns

Map of variable values and action names.

5.65.3.13 getReducedStrategy()

```
vector< tuple< vector< tuple< bool, string > >, string > > VerificationIterative::get↩
ReducedStrategy ( )
```

Takes natural strategy from memory and reduces it using reduceStrategy().

Returns

Vector of tuples <vector of tuples <variable value, variable name>, action name>.

5.65.3.14 getStrategyComplexity()

```
int VerificationIterative::getStrategyComplexity ( )
```

Natural strategy complexity before reduction. Each variable, negation, and, or are treated as value 1.

Returns

Total sum of the natural strategy complexity before reduction.

5.65.3.15 globalStateToValueBits()

```
bitset< STRATEGY_BITS > VerificationIterative::globalStateToValueBits (
            GlobalState * globalState )  [protected]
```

Changes globalState variables to a bitset used in natural strategy synthesis.

Parameters

globalState	State that we want to extractvariable values from.

Returns

Bitset of environment variable values.

5.65.3.16 isAgentInCoalition()

```
bool VerificationIterative::isAgentInCoalition (
            Agent * agent )  [protected]
```

Checks if the Agent is in a coalition based on the formula in a GlobalModelGenerator.

Parameters

agent	Pointer to an Agent that is to be checked.

Returns

Returns true if the Agent is in a coalition, otherwise returns false.

5.65.3.17 isGlobalTransitionControlledByCoalition()

```
bool VerificationIterative::isGlobalTransitionControlledByCoalition (
            GlobalTransition * globalTransition )  [protected]
```

Checks if any of the LocalTransition in a given GlobalTransition has an Agent in a coalition in the formula.

Parameters

globalTransition	Pointer to a GlobalTransition in a model.

Returns

Returns true if the Agent is in coalition in the formula, otherwise returns false.

5.65.3.18 reduceStrategy()

```
vector< tuple< vector< tuple< bool, string > >, string > > VerificationIterative::reduce↩
Strategy (
            vector< tuple< vector< tuple< bool, string >>, string >> strategyEntries,
            short lockedColumn = 0,
            bool upperHalf = false )  [protected]
```

Natural strategy reduction algorithm.

Parameters

strategyEntries	Vector of tuples <vector of tuples <variable value, variable name>, action name> containing not reduced natural strategy.
lockedColumn	Index of the furthest leftmost locked column.
upperHalf	Indicates if the first columns of the processed strategy entries can't be shuffled. True if they can't be shuffled, false otherwise.

Returns

Vector of tuples <vector of tuples <variable value, variable name>, action name>.

5.65.3.19 revertToLastDecision()

```
bool VerificationIterative::revertToLastDecision ( ) [protected]
```

Reverts the taken actions to the paint of the last decision.

Returns

True if reverted correctly, false otherwise.

5.65.3.20 testForAndFixBadAgents()

```
bool VerificationIterative::testForAndFixBadAgents (
            StateVerificationInfo * stateVerificationInfo ) [protected]
```

Fixes potentially blocking agent actions.

Parameters

stateVerificationInfo	Information for the state verification.

Returns

True if changed transitions into uncontrolled, false otherwise.

5.65.3.21 verify()

```
Result VerificationIterative::verify ( )
```

Verifies a strategy with the previously given models and formula.

Returns

Result containing data of the verification result.

5.65.3.22 verifyLocalStates()

```
bool VerificationIterative::verifyLocalStates (
            vector< LocalState * > * localStates,
            GlobalState * globalState ) [protected]
```

Verifies a set of LocalState that a GlobalState is composed of with a hardcoded formula.

Parameters

localStates	A pointer to a set of pointers to LocalState.

Returns

Returns true if there is a LocalState with a specific set of values, fulfilling the criteria, otherwise returns false.

The documentation for this class was generated from the following files:

- VerificationIterative.hpp
- VerificationIterative.cpp

5.66 yy_buffer_state Struct Reference

Public Attributes

- FILE * **yy_input_file**
- char * **yy_ch_buf**
- char * **yy_buf_pos**
- int **yy_buf_size**
- int **yy_n_chars**
- int **yy_is_our_buffer**
- int **yy_is_interactive**
- int **yy_at_bol**
- int yy_bs_lineno
- int yy_bs_column
- int **yy_fill_buffer**
- int **yy_buffer_status**

5.66.1 Member Data Documentation

5.66.1.1 yy_bs_column

```
int yy_buffer_state::yy_bs_column
```

The column count.

5.66.1.2 yy_bs_lineno

```
int yy_buffer_state::yy_bs_lineno
```

The line count.

The documentation for this struct was generated from the following files:

- scanner.c
- strategyScanner.c

5.67 yy_trans_info Struct Reference

Public Attributes

- flex_int32_t **yy_verify**
- flex_int32_t **yy_nxt**

The documentation for this struct was generated from the following files:

- scanner.c
- strategyScanner.c

5.68 yyalloc Union Reference

Public Attributes

- yy_state_t **yyss_alloc**
- YYSTYPE **yyvs_alloc**

The documentation for this union was generated from the following files:

- parser.c
- strategyParser.c

5.69 YYSTYPE Union Reference

Public Attributes

- set< class AgentTemplate * > * **model**
- class ExprNode * **expr**
- class ProbNode * **prob**
- class Assignment * **assign**
- class TransitionTemplate * **trans**
- class AgentTemplate * **agent**
- set< class Assignment * > * **assignSet**
- char * **ident**
- set< string > * **identSet**
- int **val**
- float **floatno**
- vector< class ExprNode * > * **condList**
- char * **expr**
- vector< string > * **identList**

The documentation for this union was generated from the following files:

- parser.h
- strategyParser.h

Chapter 6

File Documentation

6.1 Agent.cpp File Reference

Class of an agent. Class of an agent.

```
#include "Agent.hpp"
#include "LocalState.hpp"
```

6.1.1 Detailed Description

Class of an agent. Class of an agent.

6.2 Agent.hpp File Reference

Class of an agent. Class of an agent.

```
#include "Common.hpp"
```

Classes

- class Agent

  Contains all data for a single Agent, including id, name and all of the agents' variables.

6.2.1 Detailed Description

Class of an agent. Class of an agent.

6.3 Common.hpp File Reference

Contains all commonly used classes and structs. Contains all commonly used classes and structs.

```
#include <map>
#include <set>
#include <stack>
#include <string>
#include <utility>
#include <vector>
#include <atomic>
#include "reader/expressions.hpp"
#include "enums/GlobalStateVerificationStatus.hpp"
#include "enums/ConditionOperator.hpp"
```

Classes

- struct Cfg

6.3.1 Detailed Description

Contains all commonly used classes and structs. Contains all commonly used classes and structs.

6.4 DotGraph.cpp File Reference

Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax.

```
#include "DotGraph.hpp"
#include <algorithm>
```

6.4.1 Detailed Description

Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax.

6.5 DotGraph.hpp File Reference

Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax.

```
#include "Types.hpp"
#include "Utils.hpp"
#include "reader/nodes.hpp"
#include <string>
```

Classes

- class DotGraph

Enumerations

- enum DotGraphBase { LOCAL_MODEL , GLOBAL_MODEL , AGENT_TEMPLATE }

6.5.1 Detailed Description

Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax.

6.5.2 Enumeration Type Documentation

6.5.2.1 DotGraphBase

```
enum DotGraphBase
```

Enumerator

LOCAL_MODEL	(unfolded) local model = TS with states and transitions
GLOBAL_MODEL	(unfolded) global model = TS with states and transitions
AGENT_TEMPLATE	(folded/compact) agent graph = sets of locations and labelled edges

6.6 EpistemicClass.hpp File Reference

Struct of an epistemic class. Struct of an epistemic class.

```
#include "Common.hpp"
```

Classes

- struct EpistemicClass

  Represents a single epistemic class.

6.6.1 Detailed Description

Struct of an epistemic class. Struct of an epistemic class.

6.7 expressions.cc File Reference

Eval and helper class for expressions. Eval and helper class for expressions.

```
#include "expressions.hpp"
#include "../GlobalModelGenerator.hpp"
#include <bits/stdc++.h>
```

6.7.1 Detailed Description

Eval and helper class for expressions. Eval and helper class for expressions.

6.8 expressions.hpp File Reference

Eval and helper class for expressions. Eval and helper class for expressions.

```
#include <string>
#include <map>
#include <vector>
```

Classes

- class ExprNode

  Base node for expressions.
- class ExprConst

  Node for a constant.
- class ExprIdent

  Node for an identifier.
- class ExprAdd

  Node for addition.
- class ExprSub

  Node for subtraction.
- class ExprMul

  Node for multiplication.
- class ExprDiv

  Node for division.
- class ExprRem

  Node for modulo.
- class ExprAnd

  Node for AND operator.
- class ExprOr

  Node for OR operator.
- class ExprNot

  Node for NOT operator.
- class ExprEq

  Node for "==" operator.

- class ExprNe

  Node for "!=" operator.
- class ExprLt

  Node for "<" operator.
- class ExprLe

  Node for "<=" operator.
- class ExprGt

  Node for ">" operator.
- class ExprGe

  Node for ">=" operator.
- class ExprKnow

  Node for a knowledge operator.
- class ExprHart

  Node for a Hartley operator.
- class ProbNode

  Base node for probability calculations.
- class ProbConst

  Node for a constant.
- class ProbAdd

  Node for addition.
- class ProbSub

  Node for subtraction.
- class ProbMul

  Node for multiplication.
- class ProbDiv

  Node for division.

Typedefs

- typedef map< string, int > Environment

  Variable names with their values.

6.8.1 Detailed Description

Eval and helper class for expressions. Eval and helper class for expressions.

6.9 GlobalModel.hpp File Reference

Struct of a global model. Struct of a global model.

```
#include "Common.hpp"
```

Classes

- struct GlobalModel

  Represents a global model, containing agents and a formula.

6.9.1 Detailed Description

Struct of a global model. Struct of a global model.

6.10 GlobalModelGenerator.cpp File Reference

Generator of a global model. Class for initializing and generating a global model.

```
#include "GlobalModelGenerator.hpp"
#include "Types.hpp"
#include "Constants.hpp"
#include <algorithm>
#include <string.h>
#include <iostream>
#include <sstream>
```

Variables

- Cfg **config**

6.10.1 Detailed Description

Generator of a global model. Class for initializing and generating a global model.

6.11 GlobalModelGenerator.hpp File Reference

Generator of a global model. Class for initializing and generating a global model.

```
#include "Constants.hpp"
#include "GlobalState.hpp"
#include "GlobalTransition.hpp"
#include "Agent.hpp"
#include "Types.hpp"
#include <unordered_set>
#include <unordered_map>
#include <sstream>
```

Classes

- class GlobalModelGenerator

  Stores the local models, formula and a global model.
- struct GlobalModelGenerator::GlobalStateTupleHash

6.11.1 Detailed Description

Generator of a global model. Class for initializing and generating a global model.

6.12 GlobalState.cpp File Reference

Struct representing a global state. Struct representing a global state.

```
#include "GlobalState.hpp"
#include "LocalState.hpp"
```

6.12.1 Detailed Description

Struct representing a global state. Struct representing a global state.

6.13 GlobalState.hpp File Reference

Struct representing a global state. Struct representing a global state.

```
#include "Common.hpp"
#include "TypesDependency.hpp"
```

Classes

- struct GlobalState

  Represents a single global state.

Variables

- Cfg **config**

6.13.1 Detailed Description

Struct representing a global state. Struct representing a global state.

6.14 GlobalTransition.cpp File Reference

Struct representing a global transition. Struct representing a global transition.

```
#include "GlobalTransition.hpp"
#include "LocalTransition.hpp"
```

6.14.1 Detailed Description

Struct representing a global transition. Struct representing a global transition.

6.15 GlobalTransition.hpp File Reference

Struct representing a global transition. Struct representing a global transition.

```
#include "Common.hpp"
```

Classes

- struct GlobalTransition

  Represents a single global transition.

6.15.1 Detailed Description

Struct representing a global transition. Struct representing a global transition.

6.16 LocalState.cpp File Reference

Class representing a local state. Class representing a local state.

```
#include "LocalState.hpp"
#include "Agent.hpp"
```

6.16.1 Detailed Description

Class representing a local state. Class representing a local state.

6.17 LocalState.hpp File Reference

Class representing a local state. Class representing a local state.

```
#include "Common.hpp"
```

Classes

- class LocalState

  Represents a single LocalState, containing id, name and internal variables.

6.17.1 Detailed Description

Class representing a local state. Class representing a local state.

6.18 LocalTransition.hpp File Reference

Class representing a local transition. Class representing a local transition.

```
#include "Common.hpp"
```

Classes

- struct LocalTransition

  Represents a single local transition, containing id, global name, local name, is shared and count of the appearances.

6.18.1 Detailed Description

Class representing a local transition. Class representing a local transition.

6.19 ModelParser.cc File Reference

A model parser. A parser for converting a text file into a model.

```
#include "ModelParser.hpp"
#include "reader/nodes.hpp"
#include <stdio.h>
#include <string.h>
#include <tuple>
#include <iostream>
```

Functions

- int **yyparse** ()
- void **yyrestart** (FILE ∗)
- void **set_input_string** (const char ∗in)

Variables

- set< AgentTemplate ∗ > ∗ **modelDescription**
- FormulaTemplate **formulaDescription**

6.19.1 Detailed Description

A model parser. A parser for converting a text file into a model.

6.20 nodes.cc File Reference

Parser templates. Class for setting up a new objects from a parser.

```
#include "expressions.hpp"
#include "nodes.hpp"
#include <queue>
#include <fstream>
#include <cstring>
```

6.20.1 Detailed Description

Parser templates. Class for setting up a new objects from a parser.

6.21 nodes.hpp File Reference

Parser templates. Class for setting up a new objects from a parser.

```
#include <string>
#include <set>
#include <map>
#include "expressions.hpp"
#include "../Types.hpp"
```

Classes

- class Assignment

  Represents an assingment.
- class TransitionTemplate

  Represents a meta-transition.
- class LocalStateTemplate

  A template for the local state.
- class AgentTemplate

  Represents a single agent loaded from the description from a file.

6.21.1 Detailed Description

Parser templates. Class for setting up a new objects from a parser.

6.22 strategyNodes.hpp File Reference

Strategy parser templates. Class for setting up a new objects from a parser.

```
#include <string>
#include <set>
#include <map>
#include "../Types.hpp"
```

Classes

- class ActionTemplate

  Represents a selected action.
- class StrategyCollectionTemplate

6.22.1 Detailed Description

Strategy parser templates. Class for setting up a new objects from a parser.

6.23 StrategyParser.cc File Reference

A strategy file parser. A parser for converting a text file into a strategy.

```
#include "StrategyParser.hpp"
#include "strategyReader/strategyNodes.hpp"
#include <stdio.h>
#include <string.h>
#include <tuple>
#include <iostream>
```

Functions

- int **stratparse** ()
- void **stratrestart** (FILE ∗)
- void **set_input_string_strat** (const char ∗in)

Variables

- StrategyCollectionTemplate **strategyCollectionTemplate**

6.23.1 Detailed Description

A strategy file parser. A parser for converting a text file into a strategy.

6.24 StrategyParser.hpp File Reference

Strategy file parser. A parser for converting a text file into a strategy.

```
#include "Types.hpp"
#include <tuple>
```

Classes

- class StrategyParser

  A parser for converting a text file into a strategy.

6.24.1 Detailed Description

Strategy file parser. A parser for converting a text file into a strategy.

6.25 Types.hpp File Reference

Custom data structures. Data structures and classes containing model data.

```
#include <map>
#include <set>
#include <stack>
#include <queue>
#include <string>
#include <utility>
#include <vector>
#include "LocalState.hpp"
#include "LocalTransition.hpp"
#include "GlobalState.hpp"
#include "GlobalTransition.hpp"
#include "GlobalModel.hpp"
#include "EpistemicClass.hpp"
#include "Agent.hpp"
#include "reader/expressions.hpp"
#include <atomic>
#include "TypesDependency.hpp"
```

Classes

- struct Var

  Represents a variable in the model, containing name, initial value and persistence.
- struct Condition

  Represents a condition for LocalTransition.
- struct FormulaTemplate

  Contains a template for coalition of Agent as string from the formula.
- struct Formula

  Contains the processed verification formula.
- struct LocalModels

  Represents a single local model, contains all agents and variables.
- struct Result

Enumerations

- enum ProbabilitySign {
  NONE , EQ , NE , GT ,
  GE , LT , LE }

  Probability inequality sign used for the probabilistic verification.
- enum **VerificationFormulaMode** { **NOT_SET** = 0 , **F** = 1 , **G** = 2 }

6.25.1 Detailed Description

Custom data structures. Data structures and classes containing model data.

6.25.2 Enumeration Type Documentation

6.25.2.1 ProbabilitySign

```
enum ProbabilitySign
```

Probability inequality sign used for the probabilistic verification.

Enumerator

NONE	No probability detected.
EQ	Equal (==)
NE	Not equal (!=)
GT	Greater than (>)
GE	Greater equal (>=)
LT	Less than (<)
LE	Less equal (<=)

6.26 TypesDependency.hpp File Reference

Custom data structures, using other custom data structures. Data structures and classes containing model data that are using other custom data structures.

```
#include <bitset>
#include <queue>
```

Classes

- struct StrategyEntry
- struct StrategyBitsComparator

A comparator for two bitsets containing values for the global states.

- struct ProbabilityTrueFalse

  Contains probability of a state being in a true or false state.
- class ProbabilityEntry

  Class for handling the probability operations during probability verification.
- struct StateVerificationInfo

  Handles all single state verification information during probabilistic verification.
- struct Action

  Single action template for probabilistic verification.
- class StrategyCollection

  Handles currently selected strategy when the strategy is set.
- class ProbabilityStrategyDecisions

  Structure for the iterative set strategy generation.

Macros

- #define **STRATEGY_BITS** 64

Enumerations

- enum **VerifResult** { **NOT_VERIFIED** = 0 , **TRUE** = 1 , **FALSE** = 2 }
- enum StrategyEntryType { NOT_MODIFIED , ADDED }

  StrategyEntry entry type.
- enum HistoryEntryType {
  DECISION , STATE_STATUS , CONTEXT , MARK_DECISION_AS_INVALID ,
  UNCONTROLLED_DECISION , PROBABILITY , ANSWER_PROBABILITY }

  HistoryEntry entry type.
- enum ProbabilityCalculationType { **SUM_PROBABILITY** , **MIN_PROBABILITY** , **NONE_PROBABILITY** }

  Contains the type of the probability calculation in a given node.

6.26.1 Detailed Description

Custom data structures, using other custom data structures. Data structures and classes containing model data that are using other custom data structures.

6.26.2 Enumeration Type Documentation

6.26.2.1 HistoryEntryType

```
enum HistoryEntryType
```

HistoryEntry entry type.

Enumerator

DECISION	Made the decision to go to a state using a transition.
STATE_STATUS	Changed verification status.
CONTEXT	Recursion has gone deeper.
MARK_DECISION_AS_INVALID	Marking a transition as invalid.
UNCONTROLLED_DECISION	One uncontrolled choice from a set, from which all of them has to be OK.
PROBABILITY	There's a change in probability.
ANSWER_PROBABILITY	There's a change in the answer probability.

6.26.2.2 StrategyEntryType

```
enum StrategyEntryType
```

StrategyEntry entry type.

Enumerator

NOT_MODIFIED	Entry didn't have to get modified.
ADDED	Entry got added to the map.

6.27 Utils.cpp File Reference

Utility functions. A collection of utility functions to use in the project.

```
#include "Utils.hpp"
#include <fstream>
#include "GlobalModelGenerator.hpp"
#include <map>
#include <algorithm>
#include <iostream>
#include <cstdio>
```

Functions

- string envToString (map< string, int > env)

  Converts a map of string and int to a string.
- string agentToString (Agent *agt)

  Converts pointer to an Agant into a string containing name of the agent, its initial state, transitions with their local and global names, shared count and conditions.
- string localModelsToString (LocalModels *lm)

  Converts pointer to the LocalModels into a string cointaining all Agent instances from the model, initial values of the variables and names of the persistent values.
- void outputGlobalModel (GlobalModel *globalModel)

Prints the whole GlobalModel into the console. Contains global states with hashes, local states, variables inside those states, global variables, global transitions, local transitions and epistemic classes of agents.

- unsigned long **getMemCap** ()
- void **loadConfigFromFile** (string filename)
- void **loadConfigFromArgs** (int argc, char **argv)
- void tarjanVisit (LocalState *v, map< int, int > *dindex, map< int, int > *lowlink, stack< LocalState * > *stack, map< int, bool > *onstack, int *depth, vector< set< LocalState * >> *comp)

  Utility function for SCC-computatation.
- vector< set< LocalState * > > getLocalStatesSCC (Agent *agt)

  a quick implementation of a Tarjan SCC algorithm (based on DFS)
- map< LocalState *, vector< GlobalState * > > **getContextModel** (Formula *formula, LocalModels *localModels, Agent *agt)

Variables

- Cfg **config**

6.27.1 Detailed Description

Utility functions. A collection of utility functions to use in the project.

6.27.2 Function Documentation

6.27.2.1 agentToString()

```
string agentToString (
            Agent * agt )
```

Converts pointer to an Agant into a string containing name of the agent, its initial state, transitions with their local and global names, shared count and conditions.

Parameters

agt	Pointer to an Agent to parse into a string.

Returns

String containing all of Agent data.

6.27.2.2 envToString()

```
string envToString (
            map< string, int > env )
```

Converts a map of string and int to a string.

Parameters

env	Map to be converted into a string.

Returns

Returns string " (first_name, second_name, ..., last_name=int_value)"

6.27.2.3 getLocalStatesSCC()

```
vector<set<LocalState*> > getLocalStatesSCC (
            Agent * agt )
```

a quick implementation of a Tarjan SCC algorithm (based on DFS)

Parameters

agt	- an agent whose local graph will be inspected

Returns

localStates partition in a form of the vector, where each set correponds to a SCC

6.27.2.4 localModelsToString()

```
string localModelsToString (
            LocalModels * lm )
```

Converts pointer to the LocalModels into a string cointaining all Agent instances from the model, initial values of the variables and names of the persistent values.

Parameters

lm	Pointer to the local model to parse into a string.

Returns

String containing all of LocalModels data.

6.27.2.5 outputGlobalModel()

```
void outputGlobalModel (
            GlobalModel * globalModel )
```

Prints the whole GlobalModel into the console. Contains global states with hashes, local states, variables inside those states, global variables, global transitions, local transitions and epistemic classes of agents.

Parameters

globalModel	Pointer to a GlobalModel to print into the console.

6.27.2.6 tarjanVisit()

```
void tarjanVisit (
            LocalState * v,
            map< int, int > * dindex,
            map< int, int > * lowlink,
            stack< LocalState * > * stack,
            map< int, bool > * onstack,
            int * depth,
            vector< set< LocalState * >> * comp )
```

Utility function for SCC-computatation.

Parameters

v	- current vertex
dindex	- vertex.index (alt. vertex.num)
lowlink	- vertex.lowlink (by df. lowest dindex in the same scc reachable from vertex using tree edges followed by at most one back/cross edge)
stack	- holds candidates for SCC
onstack	- used as condition for back/cross-edge case
depth	- next available (discovery) index
comp	- result of SCC partitioning

6.28 Utils.hpp File Reference

```
#include "Types.hpp"
#include "Constants.hpp"
#include <map>
#include <string>
#include <unistd.h>
#include <sys/time.h>
#include <iostream>
#include <fstream>
```

Functions

- string envToString (map< string, int > env)

Converts a map of string and int to a string.

- string agentToString (Agent ∗agt)

  Converts pointer to an Agant into a string containing name of the agent, its initial state, transitions with their local and global names, shared count and conditions.

- string localModelsToString (LocalModels ∗lm)

  Converts pointer to the LocalModels into a string cointaining all Agent instances from the model, initial values of the variables and names of the persistent values.

- void outputGlobalModel (GlobalModel ∗globalModel)

  Prints the whole GlobalModel into the console. Contains global states with hashes, local states, variables inside those states, global variables, global transitions, local transitions and epistemic classes of agents.

- unsigned long **getMemCap** ()
- vector< set< LocalState ∗ > > getLocalStatesSCC (Agent ∗agt)

  a quick implementation of a Tarjan SCC algorithm (based on DFS)

- map< LocalState ∗, vector< GlobalState ∗ > > **getContextModel** (Formula ∗formula, LocalModels ∗localModels, Agent ∗agt)
- void **loadConfigFromFile** (string filename="config.txt")
- void **loadConfigFromArgs** (int argc, char ∗∗argv)

6.28.1 Function Documentation

6.28.1.1 agentToString()

```
string agentToString (
            Agent * agt )
```

Converts pointer to an Agant into a string containing name of the agent, its initial state, transitions with their local and global names, shared count and conditions.

Parameters

agt	Pointer to an Agent to parse into a string.

Returns

String containing all of Agent data.

6.28.1.2 envToString()

```
string envToString (
            map< string, int > env )
```

Converts a map of string and int to a string.

Parameters

env	Map to be converted into a string.

Returns

Returns string " (first_name, second_name, ..., last_name=int_value)"

6.28.1.3 getLocalStatesSCC()

```
vector<set<LocalState*> > getLocalStatesSCC (
            Agent * agt )
```

a quick implementation of a Tarjan SCC algorithm (based on DFS)

Parameters

agt	- an agent whose local graph will be inspected

Returns

localStates partition in a form of the vector, where each set correponds to a SCC

6.28.1.4 localModelsToString()

```
string localModelsToString (
            LocalModels * lm )
```

Converts pointer to the LocalModels into a string cointaining all Agent instances from the model, initial values of the variables and names of the persistent values.

Parameters

lm	Pointer to the local model to parse into a string.

Returns

String containing all of LocalModels data.

6.28.1.5 outputGlobalModel()

```
void outputGlobalModel (
            GlobalModel * globalModel )
```

Prints the whole GlobalModel into the console. Contains global states with hashes, local states, variables inside those states, global variables, global transitions, local transitions and epistemic classes of agents.

Parameters

globalModel	Pointer to a GlobalModel to print into the console.

6.29 Verification.cpp File Reference

Class for verification of the formula on a model. Class for verification of the specified formula on a specified model.

```
#include "Verification.hpp"
#include <bits/stdc++.h>
```

Macros

- #define **DEPTH_PREFIX** string(depth ∗ 4, ' ')

Functions

- string verStatusToStr (GlobalStateVerificationStatus status)

  Converts global verification status into a string.
- void dbgVerifStatus (string prefix, GlobalState ∗gs, GlobalStateVerificationStatus st, string reason)

  Print a debug message of a verification status to the console.
- void dbgHistEnt (string prefix, HistoryEntry ∗h)

  Print a single debug message with a history entry to the console.

Variables

- Cfg **config**

6.29.1 Detailed Description

Class for verification of the formula on a model. Class for verification of the specified formula on a specified model.

6.29.2 Function Documentation

6.29.2.1 dbgHistEnt()

```
void dbgHistEnt (
            string prefix,
            HistoryEntry * h )
```

Print a single debug message with a history entry to the console.

Parameters

prefix	A prefix string to append to the front of the entry.
h	A pointer to the HistoryEntry struct which will be printed out.

6.29.2.2 dbgVerifStatus()

```
void dbgVerifStatus (
            string prefix,
            GlobalState * gs,
            GlobalStateVerificationStatus st,
            string reason )
```

Print a debug message of a verification status to the console.

Parameters

prefix	A prefix string to append to the front of every entry.
gs	Pointer to a GlobalState.
st	Enum with a verification status of a global state.
reason	String with a reason why the function was called, e.g. "entered state", "all passed".

6.29.2.3 verStatusToStr()

```
string verStatusToStr (
            GlobalStateVerificationStatus status )
```

Converts global verification status into a string.

Parameters

status	Enum value to be converted.

Returns

Verification status converted into a string.

6.30 Verification.hpp File Reference

```
#include <stack>
#include "Types.hpp"
#include "TypesDependency.hpp"
#include "GlobalModelGenerator.hpp"
#include <bitset>
```

Classes

- struct HistoryEntry

  Structure used to save model traversal history.
- class HistoryDbg

  Stores history and allows displaying it to the console.
- class Verification

  A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

Macros

- #define **STRATEGY_BITS** 64

Enumerations

- enum TraversalMode { NORMAL , REVERT , RESTORE }

  Current model traversal mode.

Functions

- string verStatusToStr (GlobalStateVerificationStatus status)

  Converts global verification status into a string.

6.30.1 Enumeration Type Documentation

6.30.1.1 TraversalMode

```
enum TraversalMode
```

Current model traversal mode.

Enumerator

NORMAL	Normal model traversal.
REVERT	Backtracking through recursion with state rollback.
RESTORE	Backtracking through recursion.

6.30.2 Function Documentation

6.30.2.1 verStatusToStr()

```
string verStatusToStr (
            GlobalStateVerificationStatus status )
```

Converts global verification status into a string.

Parameters

status	Enum value to be converted.

Returns

Verification status converted into a string.

6.31 VerificationIterative.cpp File Reference

Class for iteratively generated verification of the formula on a model. Class for iteratively generated verification of the specified formula on a specified model.

```
#include "VerificationIterative.hpp"
#include <bits/stdc++.h>
```

Macros

- #define **DEPTH_PREFIX** string(depth ∗ 4, ' ')

Functions

- string verifStatusToStr (GlobalStateVerificationStatus status)

  Converts global verification status into a string.

Variables

- Cfg **config**

6.31.1 Detailed Description

Class for iteratively generated verification of the formula on a model. Class for iteratively generated verification of the specified formula on a specified model.

6.31.2 Function Documentation

6.31.2.1 verifStatusToStr()

```
string verifStatusToStr (
            GlobalStateVerificationStatus status )
```

Converts global verification status into a string.

Parameters

status	Enum value to be converted.

Returns

Verification status converted into a string.

6.32 VerificationIterative.hpp File Reference

```
#include <stack>
#include "Types.hpp"
#include "TypesDependency.hpp"
#include "GlobalModelGenerator.hpp"
#include <bitset>
```

Classes

- struct DecisionEntry
- class VerificationIterative

  A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

Enumerations

- enum GraphTraversalMode { NORMAL_TRAVERSAL , RESTORE_STATES }

  Current model traversal mode.

Functions

- string verifStatusToStr (GlobalStateVerificationStatus status)

  Converts global verification status into a string.

6.32.1 Enumeration Type Documentation

6.32.1.1 GraphTraversalMode

```
enum GraphTraversalMode
```

Current model traversal mode.

Enumerator

NORMAL_TRAVERSAL	Normal model traversal.
RESTORE_STATES	Reconstructing recursion.

6.32.2 Function Documentation

6.32.2.1 verifStatusToStr()

```
string verifStatusToStr (
            GlobalStateVerificationStatus status )
```

Converts global verification status into a string.

Parameters

status	Enum value to be converted.

Returns

Verification status converted into a string.

Index

Action, 15
ActionTemplate, 15
    ActionTemplate, 16
addAction
    StrategyCollectionTemplate, 93
ADDED
    TypesDependency.hpp, 137
addEdge
    DotGraph, 27
addEntry
    HistoryDbg, 74
addHistoryAnswerProbability
    VerificationIterative, 112
addHistoryContext
    Verification, 100
    VerificationIterative, 113
addHistoryDecision
    Verification, 100
    VerificationIterative, 113
addHistoryMarkDecisionAsInvalid
    Verification, 100
    VerificationIterative, 114
addHistoryStateStatus
    Verification, 101
    VerificationIterative, 114
addHistoryUncontrolledDecision
    Verification, 101
addInitial
    AgentTemplate, 19
addLocal
    AgentTemplate, 19
addNode
    DotGraph, 27
addPersistent
    AgentTemplate, 20
addTransition
    AgentTemplate, 20
Agent, 16
    Agent, 17
    includesState, 17
Agent.cpp, 123
Agent.hpp, 123
AGENT_TEMPLATE
    DotGraph.hpp, 125
AgentTemplate, 18
    addInitial, 19
    addLocal, 19
    addPersistent, 20
    addTransition, 20
    generateAgent, 21
    setIdent, 21
    setInitState, 22
agentToString
    Utils.cpp, 138
    Utils.hpp, 141
ANSWER_PROBABILITY
    TypesDependency.hpp, 137
areGlobalStatesInTheSameEpistemicClass
    Verification, 101
    VerificationIterative, 114
assign
    Assignment, 24
Assignment, 23
    assign, 24
    Assignment, 23

Cfg, 24
checkIfProbabilistic
    VerificationIterative, 115
checkUncontrolledSet
    Verification, 102
cloneEntry
    HistoryDbg, 75
Common.hpp, 124
compare
    LocalState, 78
computeEpistemicClassHash
    GlobalModelGenerator, 64
computeGlobalStateHash
    GlobalModelGenerator, 64
Condition, 25
CONTEXT
    TypesDependency.hpp, 137
createIterativeStrategy
    GlobalModelGenerator, 64

dbgHistEnt
    Verification.cpp, 143
dbgVerifStatus
    Verification.cpp, 144
DECISION
    TypesDependency.hpp, 137
DecisionEntry, 25
dissolveLoopedProbabilityTransition
    VerificationIterative, 115
DotGraph, 26
    addEdge, 27
    addNode, 27
    saveToFile, 27

styleString, 28
DotGraph.cpp, 124
DotGraph.hpp, 124
AGENT_TEMPLATE, 125
DotGraphBase, 125
GLOBAL_MODEL, 125
LOCAL_MODEL, 125
DotGraphBase
DotGraph.hpp, 125

envToString
Utils.cpp, 138
Utils.hpp, 141
EpistemicClass, 28
EpistemicClass.hpp, 125
EQ
Types.hpp, 135
equivalentGlobalTransitions
Verification, 102
VerificationIterative, 115
eval
ExprAdd, 29
ExprAnd, 32
ExprConst, 33
ExprDiv, 34
ExprEq, 37
ExprGe, 38
ExprGt, 39
ExprHart, 42
ExprIdent, 43
ExprKnow, 46
ExprLe, 47
ExprLt, 48
ExprMul, 51
ExprNe, 52
ExprNode, 53
ExprNot, 54
ExprOr, 56
ExprRem, 57
ExprSub, 58
ProbAdd, 84
ProbConst, 86
ProbDiv, 87
ProbMul, 88
ProbNode, 89
ProbSub, 90
expandAndReduceAllStates
GlobalModelGenerator, 65
expandState
GlobalModelGenerator, 65
expandStateAndReturn
GlobalModelGenerator, 65
ExprAdd, 29
eval, 29
ExprAdd, 29
ExprAnd, 31
eval, 32
ExprAnd, 31
ExprConst, 32
eval, 33
ExprConst, 33
ExprDiv, 34
eval, 34
ExprDiv, 34
ExprEq, 36
eval, 37
ExprEq, 36
expressions.cc, 126
expressions.hpp, 126
ExprGe, 37
eval, 38
ExprGe, 38
ExprGt, 39
eval, 39
ExprGt, 39
ExprHart, 41
eval, 42
ExprHart, 41
ExprIdent, 42
eval, 43
ExprIdent, 43
ExprKnow, 45
eval, 46
ExprKnow, 45
ExprLe, 46
eval, 47
ExprLe, 47
ExprLt, 48
eval, 48
ExprLt, 48
ExprMul, 50
eval, 51
ExprMul, 50
ExprNe, 51
eval, 52
ExprNe, 52
ExprNode, 53
eval, 53
ExprNot, 54
eval, 54
ExprNot, 54
ExprOr, 55
eval, 56
ExprOr, 55
ExprRem, 56
eval, 57
ExprRem, 57
ExprSub, 58
eval, 58
ExprSub, 58

findGlobalStateInEpistemicClass
GlobalModelGenerator, 65
findLoopedProbabilityTransitions
VerificationIterative, 116
findOrCreateEpistemicClass
GlobalModelGenerator, 66
findOrCreateEpistemicClassForKnowledge

GlobalModelGenerator, 66
Formula, 60
FormulaTemplate, 60

GE
Types.hpp, 135
generateAgent
AgentTemplate, 21
generateGlobalTransitions
GlobalModelGenerator, 67
generateInitState
GlobalModelGenerator, 67
generateStateFromLocalStates
GlobalModelGenerator, 67
getActionNameFromStateInStrategy
GlobalModelGenerator, 68
getAgentInstanceByName
GlobalModelGenerator, 68
getCoalitionIdentifier
GlobalModelGenerator, 68
getCurrentGlobalModel
GlobalModelGenerator, 69
getEpistemicClassForGlobalState
Verification, 103
VerificationIterative, 116
getFormula
GlobalModelGenerator, 69
getFormulaCorectness
GlobalModelGenerator, 69
getFormulaSize
GlobalModelGenerator, 69
getGlobalStateEnvironment
GlobalState, 72
getLocalStatesSCC
Utils.cpp, 139
Utils.hpp, 142
getNaturalStrategy
Verification, 103
VerificationIterative, 116
getReducedStrategy
Verification, 103
VerificationIterative, 117
getStrategyComplexity
Verification, 103
VerificationIterative, 117
GLOBAL_MODEL
DotGraph.hpp, 125
GlobalModel, 61
GlobalModel.hpp, 127
GlobalModelGenerator, 62
computeEpistemicClassHash, 64
computeGlobalStateHash, 64
createIterativeStrategy, 64
expandAndReduceAllStates, 65
expandState, 65
expandStateAndReturn, 65
findGlobalStateInEpistemicClass, 65
findOrCreateEpistemicClass, 66
findOrCreateEpistemicClassForKnowledge, 66
generateGlobalTransitions, 67
generateInitState, 67
generateStateFromLocalStates, 67
getActionNameFromStateInStrategy, 68
getAgentInstanceByName, 68
getCoalitionIdentifier, 68
getCurrentGlobalModel, 69
getFormula, 69
getFormulaCorectness, 69
getFormulaSize, 69
initModel, 70
initStrategy, 70
nextIterativeStrategy, 70
GlobalModelGenerator.cpp, 128
GlobalModelGenerator.hpp, 128
GlobalModelGenerator::GlobalStateTupleHash, 72
GlobalState, 71
getGlobalStateEnvironment, 72
toString, 72
GlobalState.cpp, 129
GlobalState.hpp, 129
globalStateToValueBits
Verification, 104
VerificationIterative, 117
GlobalTransition, 73
joinLocalTransitionNames, 73
GlobalTransition.cpp, 129
GlobalTransition.hpp, 130
GraphTraversalMode
VerificationIterative.hpp, 147
GT
Types.hpp, 135

HistoryDbg, 74
addEntry, 74
cloneEntry, 75
markEntry, 75
print, 75
HistoryEntry, 76
toString, 76
HistoryEntryType
TypesDependency.hpp, 136

includesState
Agent, 17
increaseProbability
Verification, 104
initModel
GlobalModelGenerator, 70
initStrategy
GlobalModelGenerator, 70
isAgentInCoalition
Verification, 104
VerificationIterative, 117
isGlobalTransitionControlledByCoalition
Verification, 105
VerificationIterative, 118
IterativeVerif, 77

joinLocalTransitionNames
GlobalTransition, 73

LE
Types.hpp, 135
LOCAL_MODEL
DotGraph.hpp, 125
LocalModels, 77
localModelsToString
Utils.cpp, 139
Utils.hpp, 142
LocalState, 78
compare, 78
toString, 79
LocalState.cpp, 130
LocalState.hpp, 130
LocalStateTemplate, 79
LocalTransition, 80
LocalTransition.hpp, 131
lowerProbability
Verification, 105
LT
Types.hpp, 135

MARK_DECISION_AS_INVALID
TypesDependency.hpp, 137
markEntry
HistoryDbg, 75
ModelParser, 81
parse, 81
parseAndOverwriteFormula, 81
ModelParser.cc, 131

NE
Types.hpp, 135
newHistoryMarkDecisionAsInvalid
Verification, 105
nextIterativeStrategy
GlobalModelGenerator, 70
nodes.cc, 132
nodes.hpp, 132
NONE
Types.hpp, 135
NORMAL
Verification.hpp, 145
NORMAL_TRAVERSAL
VerificationIterative.hpp, 148
NOT_MODIFIED
TypesDependency.hpp, 137

outputGlobalModel
Utils.cpp, 139
Utils.hpp, 142

parse
ModelParser, 81
StrategyParser, 94
parseAndOverwriteFormula
ModelParser, 81

print
HistoryDbg, 75
printCurrentHistory
Verification, 106
PROBABILITY
TypesDependency.hpp, 137
ProbabilityEntry, 82
ProbabilitySign
Types.hpp, 135
ProbabilityStrategyDecisions, 83
ProbabilityTrueFalse, 83
ProbAdd, 83
eval, 84
ProbAdd, 84
ProbConst, 85
eval, 86
ProbConst, 85
ProbDiv, 86
eval, 87
ProbDiv, 87
ProbMul, 87
eval, 88
ProbMul, 88
ProbNode, 89
eval, 89
ProbSub, 90
eval, 90
ProbSub, 90

reduceStrategy
Verification, 106
VerificationIterative, 118
RESTORE
Verification.hpp, 145
RESTORE_STATES
VerificationIterative.hpp, 148
restoreHistory
Verification, 107
Result, 91
REVERT
Verification.hpp, 145
revertLastDecision
Verification, 107
revertToLastDecision
VerificationIterative, 119

saveToFile
DotGraph, 27
setIdent
AgentTemplate, 21
setInitState
AgentTemplate, 22
STATE_STATUS
TypesDependency.hpp, 137
StateVerificationInfo, 91
StrategyBitsComparator, 92
StrategyCollection, 92
StrategyCollectionTemplate, 93
addAction, 93

StrategyEntry, 93
StrategyEntryType
TypesDependency.hpp, 137
strategyNodes.hpp, 133
StrategyParser, 94
parse, 94
StrategyParser.cc, 133
StrategyParser.hpp, 134
styleString
DotGraph, 28

tarjanVisit
Utils.cpp, 140
testForAndFixBadAgents
VerificationIterative, 119
TestVerif, 95
toString
GlobalState, 72
HistoryEntry, 76
LocalState, 79
TransitionTemplate, 95
TransitionTemplate, 96
TraversalMode
Verification.hpp, 145
Types.hpp, 134
EQ, 135
GE, 135
GT, 135
LE, 135
LT, 135
NE, 135
NONE, 135
ProbabilitySign, 135
TypesDependency.hpp, 135
ADDED, 137
ANSWER_PROBABILITY, 137
CONTEXT, 137
DECISION, 137
HistoryEntryType, 136
MARK_DECISION_AS_INVALID, 137
NOT_MODIFIED, 137
PROBABILITY, 137
STATE_STATUS, 137
StrategyEntryType, 137
UNCONTROLLED_DECISION, 137

UNCONTROLLED_DECISION
TypesDependency.hpp, 137
undoHistoryUntil
Verification, 107
undoLastHistoryEntry
Verification, 108
Utils.cpp, 137
agentToString, 138
envToString, 138
getLocalStatesSCC, 139
localModelsToString, 139
outputGlobalModel, 139
tarjanVisit, 140
Utils.hpp, 140
agentToString, 141
envToString, 141
getLocalStatesSCC, 142
localModelsToString, 142
outputGlobalModel, 142

Var, 96
Verification, 97
addHistoryContext, 100
addHistoryDecision, 100
addHistoryMarkDecisionAsInvalid, 100
addHistoryStateStatus, 101
addHistoryUncontrolledDecision, 101
areGlobalStatesInTheSameEpistemicClass, 101
checkUncontrolledSet, 102
equivalentGlobalTransitions, 102
getEpistemicClassForGlobalState, 103
getNaturalStrategy, 103
getReducedStrategy, 103
getStrategyComplexity, 103
globalStateToValueBits, 104
increaseProbability, 104
isAgentInCoalition, 104
isGlobalTransitionControlledByCoalition, 105
lowerProbability, 105
newHistoryMarkDecisionAsInvalid, 105
printCurrentHistory, 106
reduceStrategy, 106
restoreHistory, 107
revertLastDecision, 107
undoHistoryUntil, 107
undoLastHistoryEntry, 108
Verification, 99
verify, 108
verifyGlobalState, 108
verifyLocalStates, 109
verifyStrategy, 109
verifyTransitionSets, 109
Verification.cpp, 143
dbgHistEnt, 143
dbgVerifStatus, 144
verStatusToStr, 144
Verification.hpp, 144
NORMAL, 145
RESTORE, 145
REVERT, 145
TraversalMode, 145
verStatusToStr, 145
VerificationIterative, 110
addHistoryAnswerProbability, 112
addHistoryContext, 113
addHistoryDecision, 113
addHistoryMarkDecisionAsInvalid, 114
addHistoryStateStatus, 114
areGlobalStatesInTheSameEpistemicClass, 114
checkIfProbabilistic, 115
dissolveLoopedProbabilityTransition, 115
equivalentGlobalTransitions, 115

findLoopedProbabilityTransitions, 116
getEpistemicClassForGlobalState, 116
getNaturalStrategy, 116
getReducedStrategy, 117
getStrategyComplexity, 117
globalStateToValueBits, 117
isAgentInCoalition, 117
isGlobalTransitionControlledByCoalition, 118
reduceStrategy, 118
revertToLastDecision, 119
testForAndFixBadAgents, 119
VerificationIterative, 112
verify, 119
verifyLocalStates, 119
VerificationIterative.cpp, 146
verifStatusToStr, 146
VerificationIterative.hpp, 147
GraphTraversalMode, 147
NORMAL_TRAVERSAL, 148
RESTORE_STATES, 148
verifStatusToStr, 148
verifStatusToStr
VerificationIterative.cpp, 146
VerificationIterative.hpp, 148
verify
Verification, 108
VerificationIterative, 119
verifyGlobalState
Verification, 108
verifyLocalStates
Verification, 109
VerificationIterative, 119
verifyStrategy
Verification, 109
verifyTransitionSets
Verification, 109
verStatusToStr
Verification.cpp, 144
Verification.hpp, 145

yy_bs_column
yy_buffer_state, 120
yy_bs_lineno
yy_buffer_state, 120
yy_buffer_state, 120
yy_bs_column, 120
yy_bs_lineno, 120
yy_trans_info, 121
yyalloc, 121
YYSTYPE, 121